

Data-Driven Computer Animation

Learning Human Motion

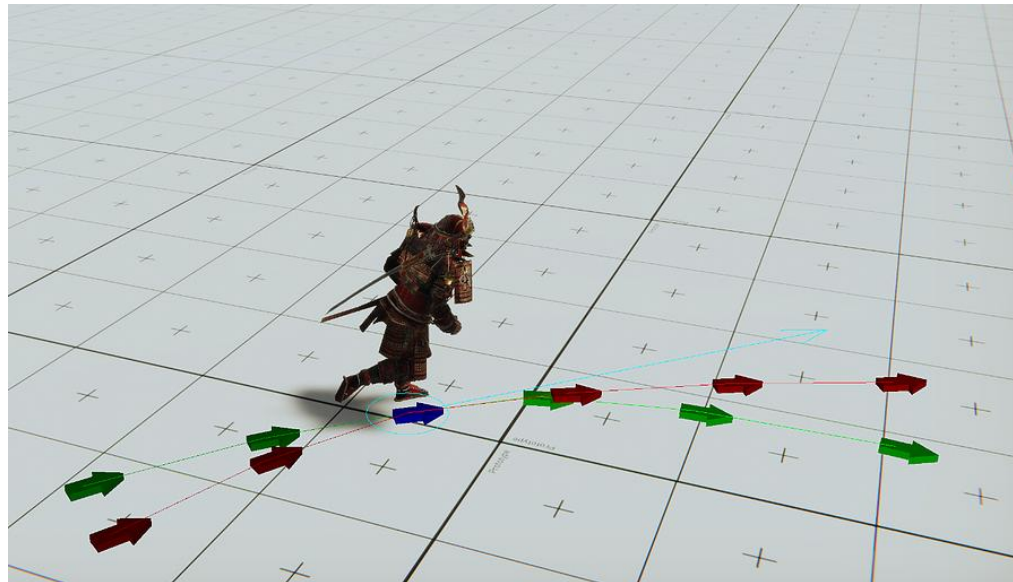
Taku Komura

Overview

- Motion Matching
- Deep learning-based motion synthesis
 - Unsupervised learning by temporal convolution
 - Supervised learning by temporal convolution
 - Etc.

Nearest Neighbor Search

- Let's think of synthesizing a series of motions given the user instructions
- What would be a good method to do this?
- The simplest approach is to search in the motion database that follows the instruction



K-Nearest Neighbors

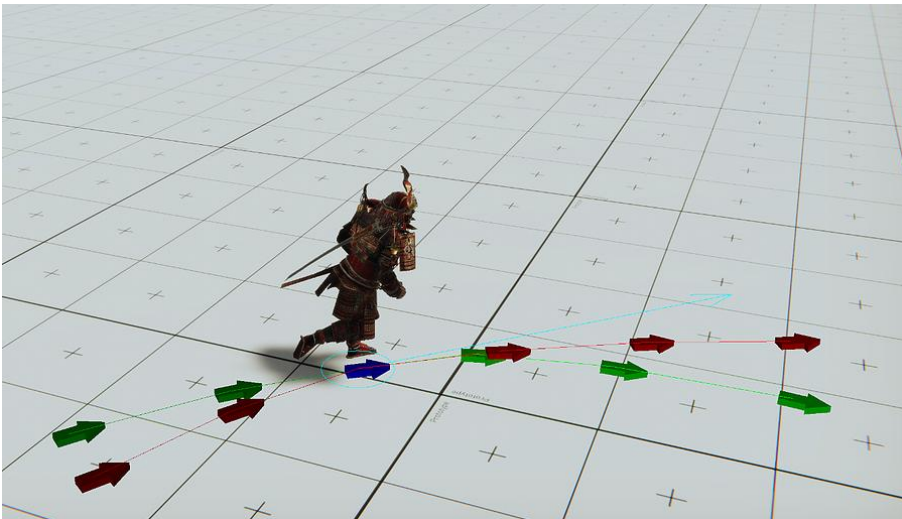
- A classic approach used for classification/regression
- We wish to predict $\mathbf{y}$, given the input $\mathbf{x}$
- We assume we have training dataset $(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_n, \mathbf{y}_n)$
- Given the current input we search for the k -samples where
$$\|\mathbf{x} - \mathbf{x}_{(1)}\| < \|\mathbf{x} - \mathbf{x}_{(2)}\| < \dots < \|\mathbf{x} - \mathbf{x}_{(k)}\|$$

We estimate the $\mathbf{y}$ at $\mathbf{x}$ by

$$\mathbf{y} = \sum_{i=1}^k \mathbf{w}_i \mathbf{y}_{(i)}$$

Motion Matching

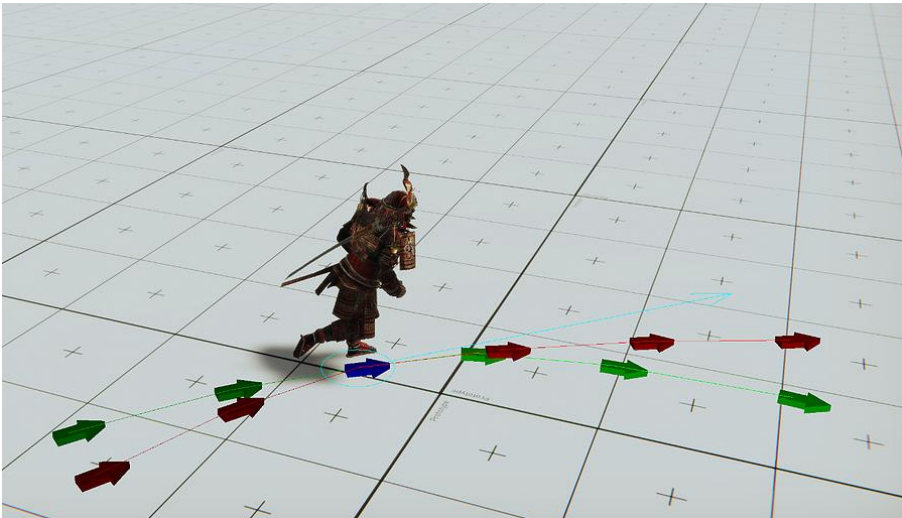
- A technique developed by Ubisoft Montreal
- Define a feature, and search in the database that matches
 - The current state of the character
 - User instruction given by the user
- Current state-of-the-art in the computer game industry



Clavet 2016, “Motion Matching and The Road to Next-Gen Animation”
Holden et al. 2020, “Learned Motion Matching”

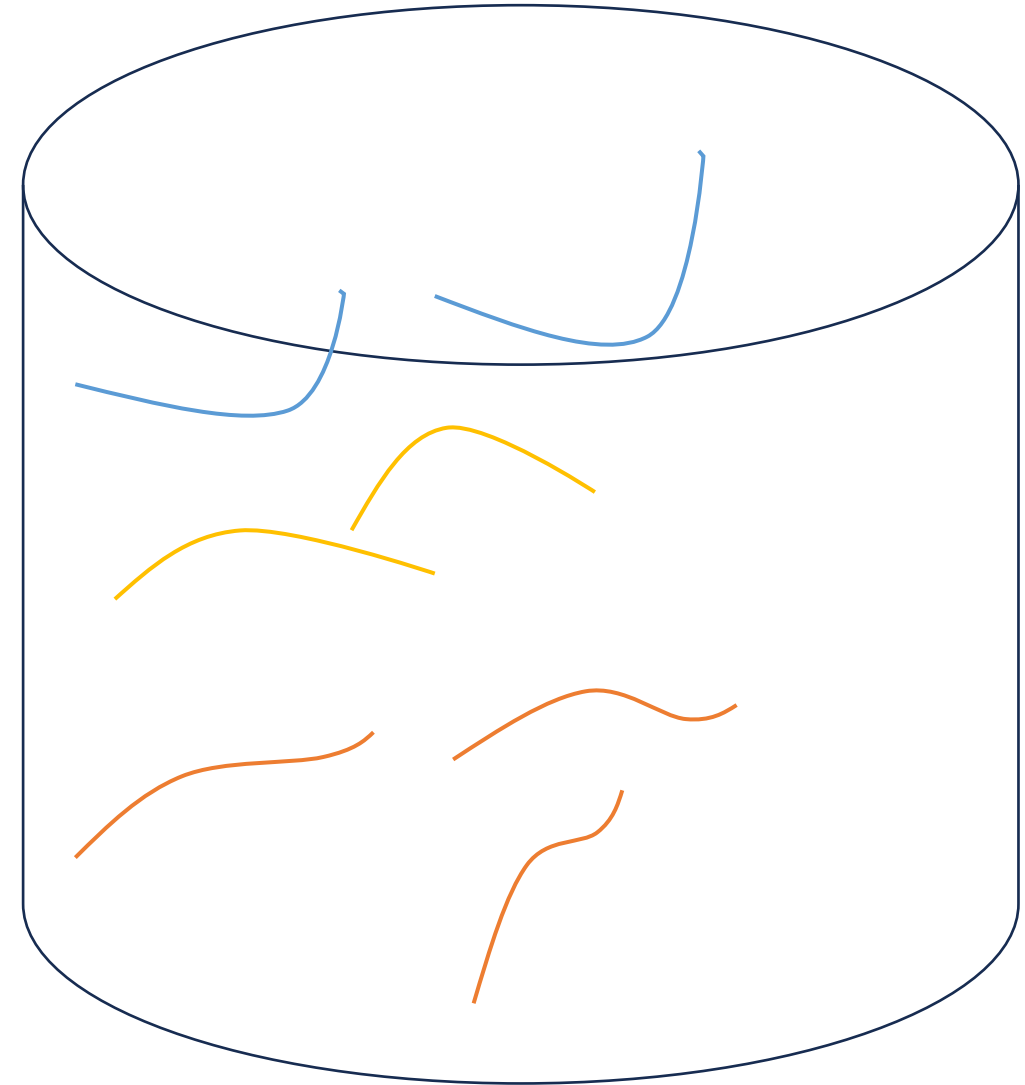
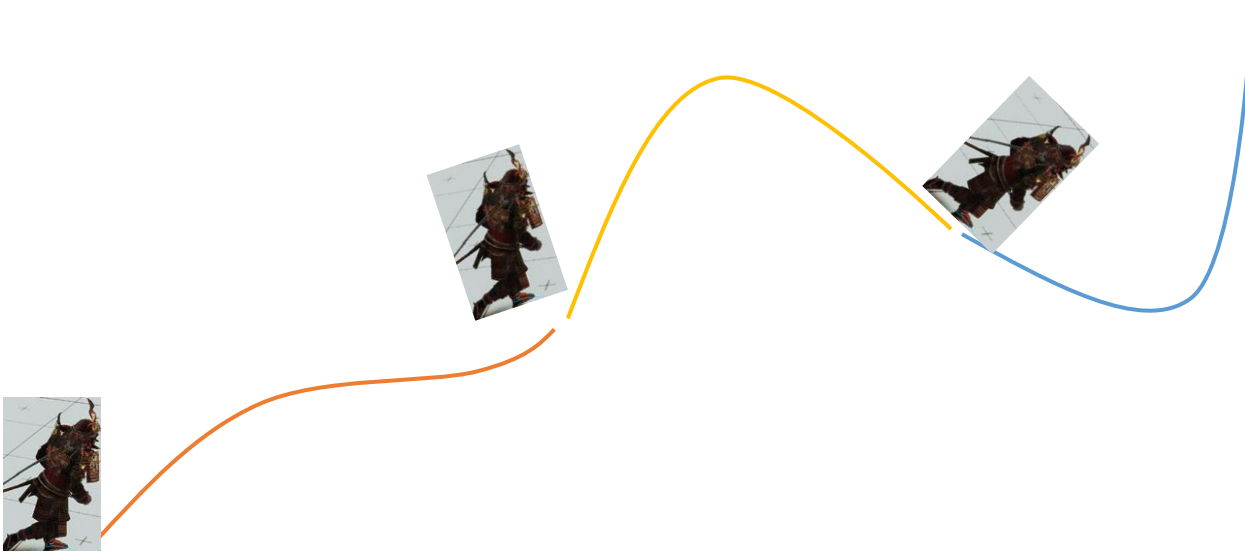
Motion Matching

- Features that are used:
 - Trajectory on the ground
 - The velocity of the feet (for bipedal locomotion like walking, running)
- The matched motions are blended with the current motion

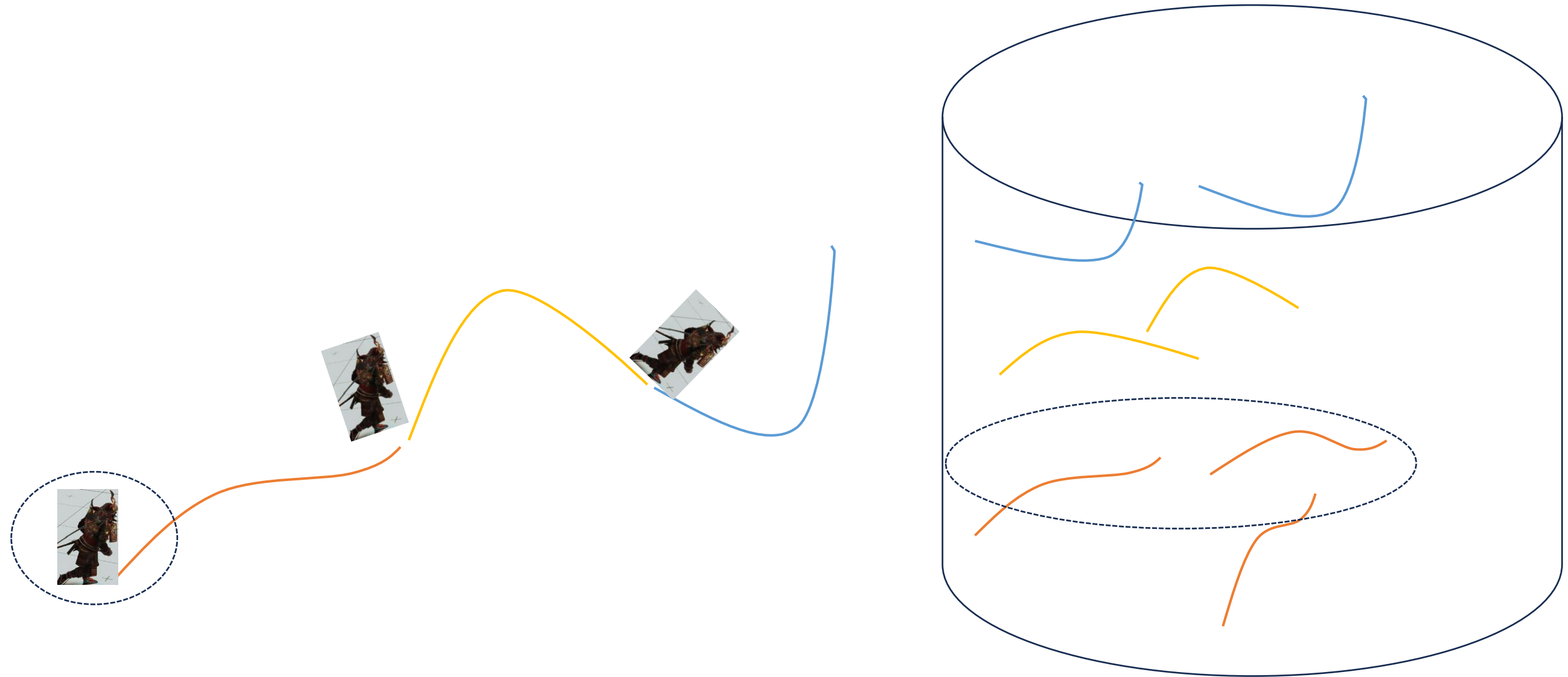


Clavet 2016, “Motion Matching and The Road to Next-Gen Animation”
Holden et al. 2020, “Learned Motion Matching”

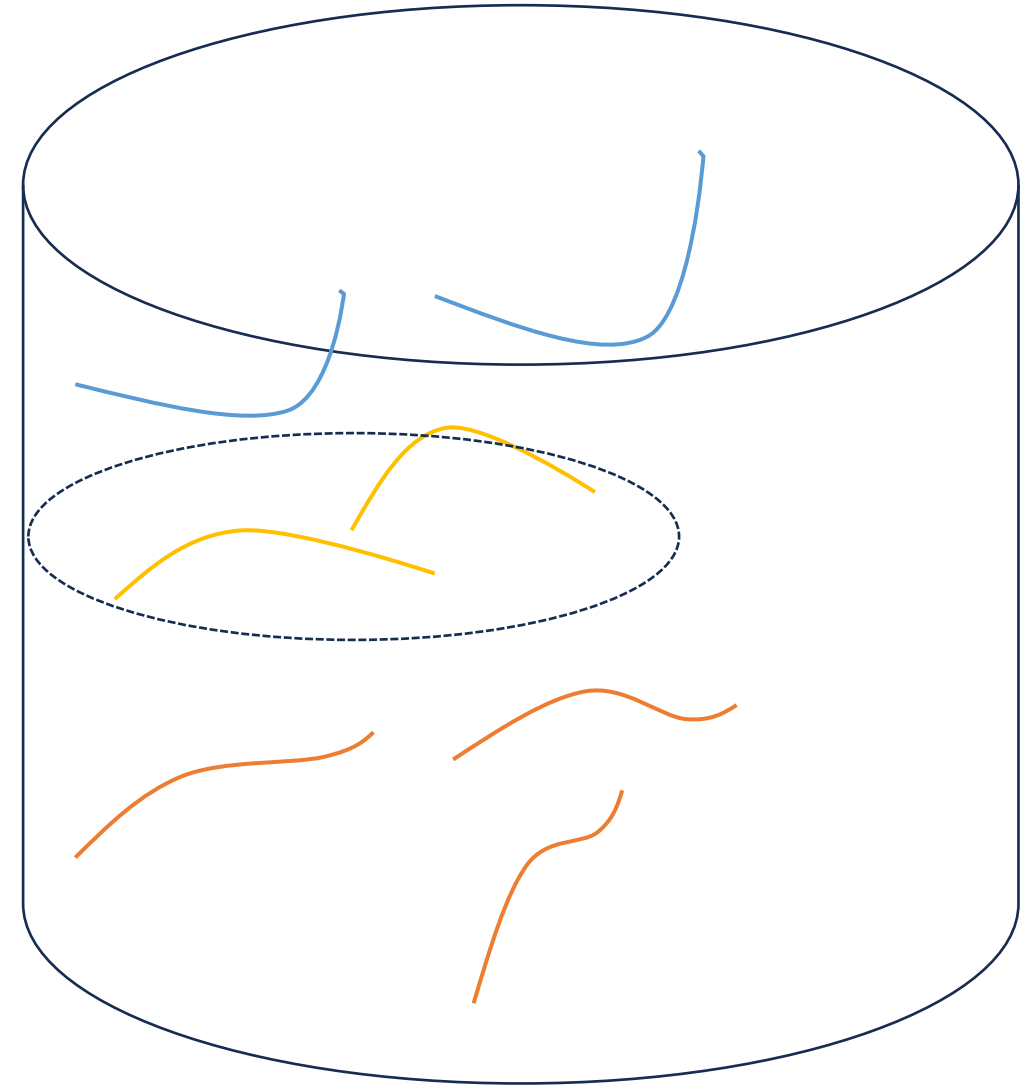
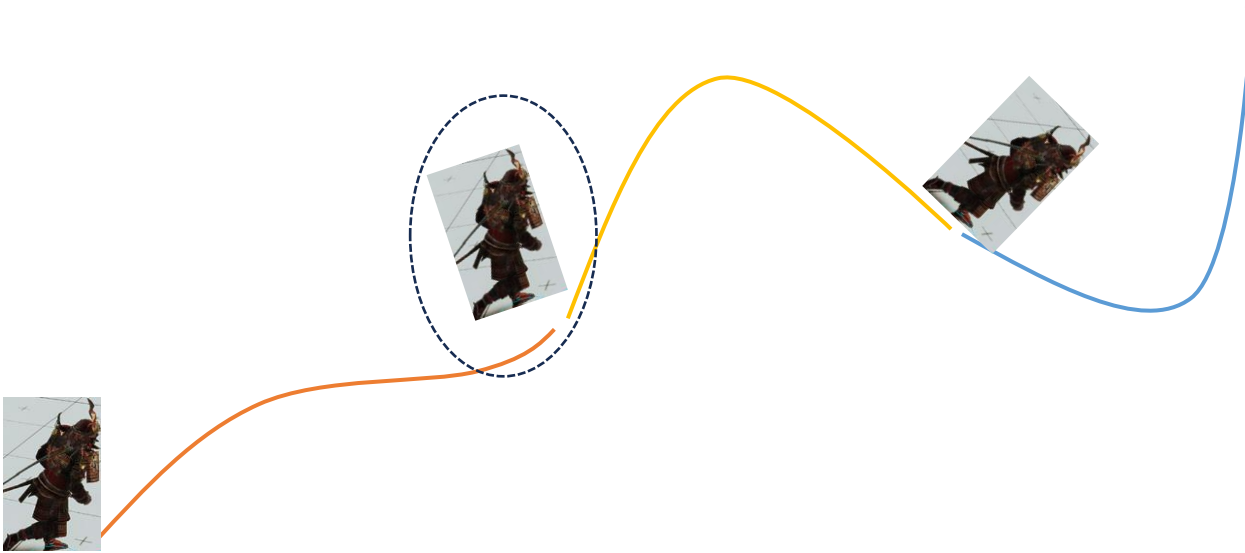
Motion database



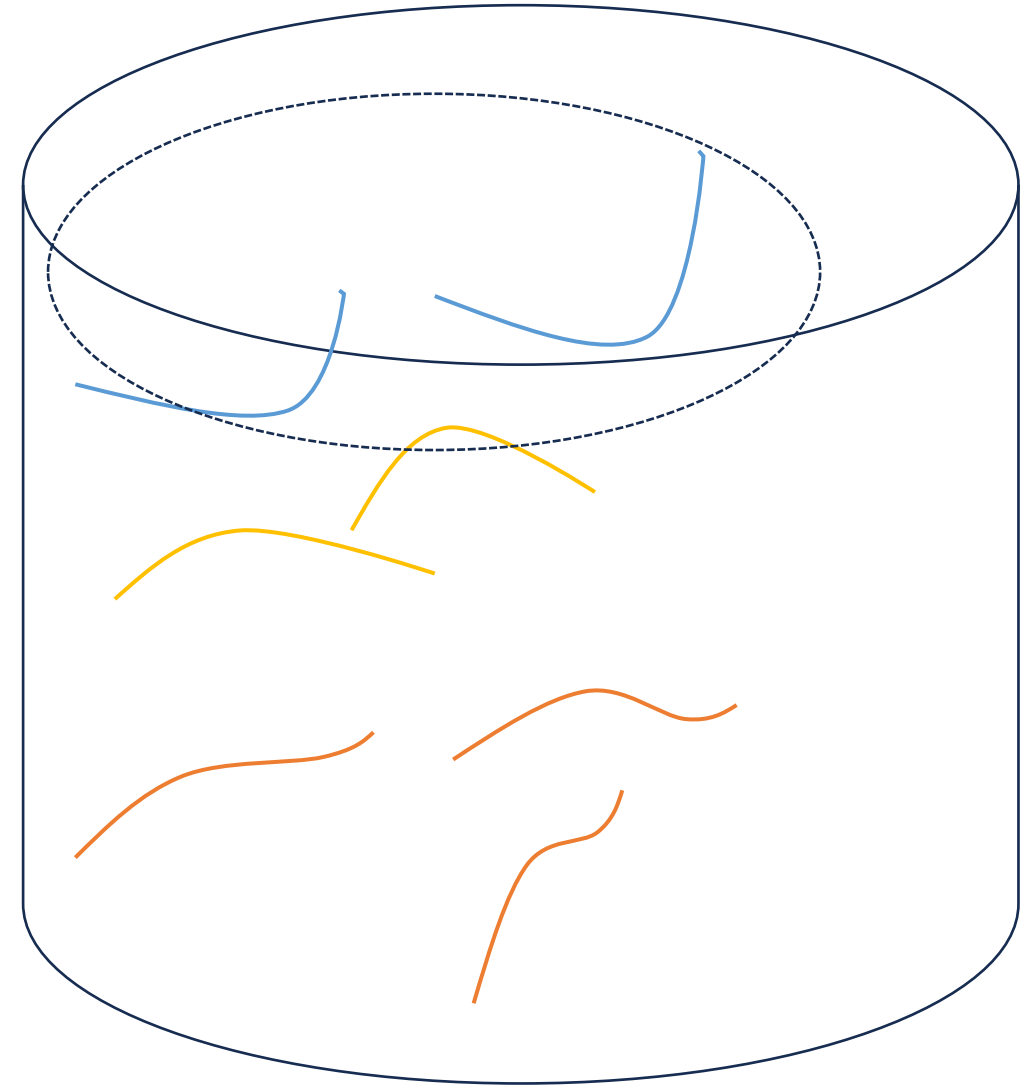
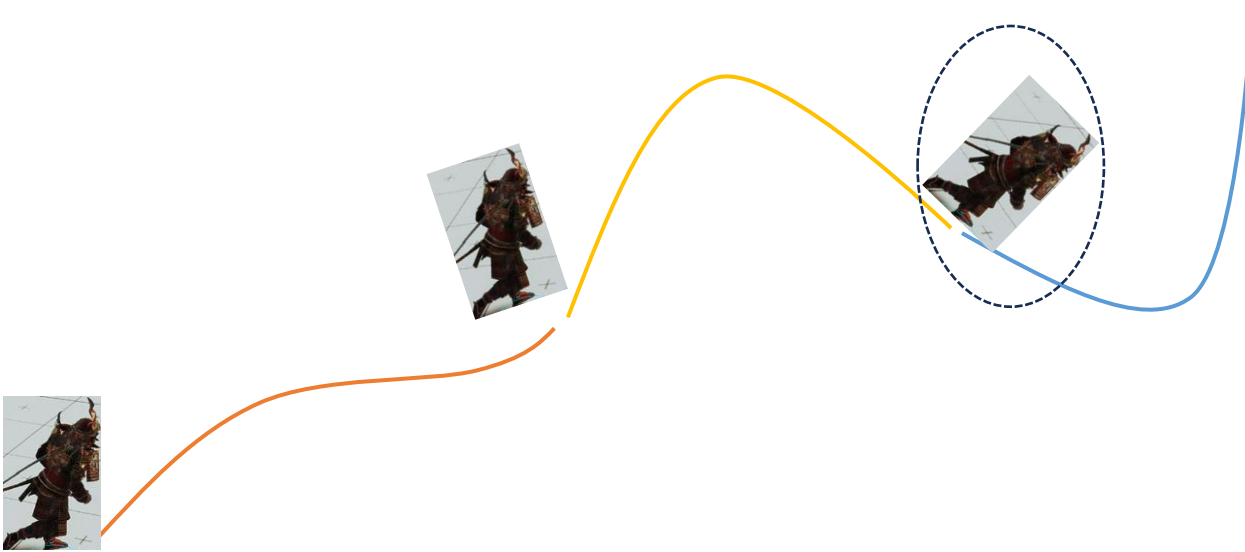
Motion database

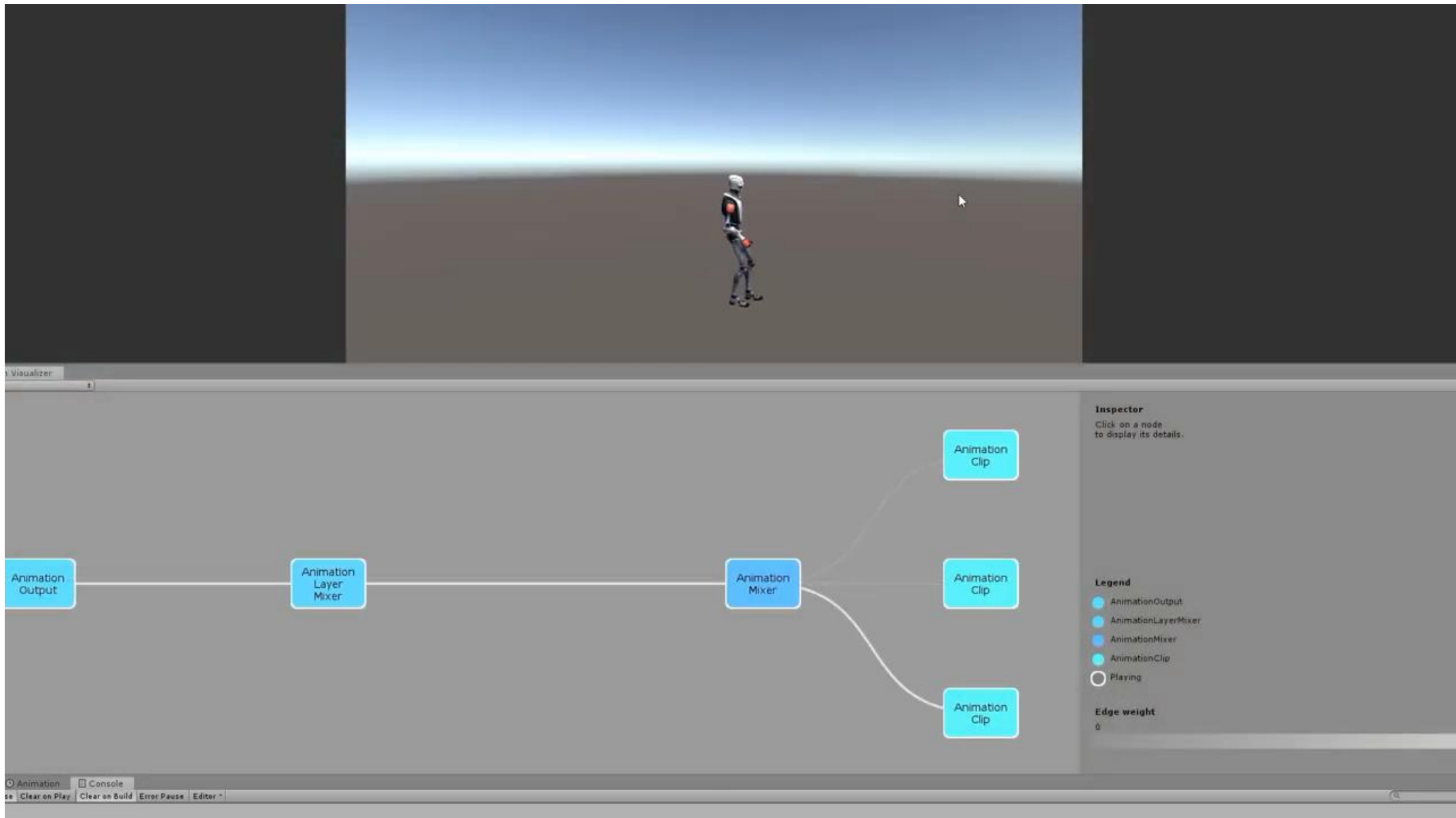


Motion database



Motion database





Motion Matching

- Advantages:
 - Quality of the motion is good as the original motion data are played
 - Motions can be added into the database whenever you wish
- Disadvantages:
 - The search time grows as the data size grows
 - The search feature must be carefully selected
 - No motions are newly generated (no learning happening)

Deep Learning

- ◆ A popular framework that can learn from a large amount of data
- ◆ Applied for speech/image recognition/synthesis
- ◆ Also, very popular for computer graphics/computer animation purposes

Deep Learning

- ◆ Automatic Colorization



Colorado National Park, 1941



Textile Mill, June 1937



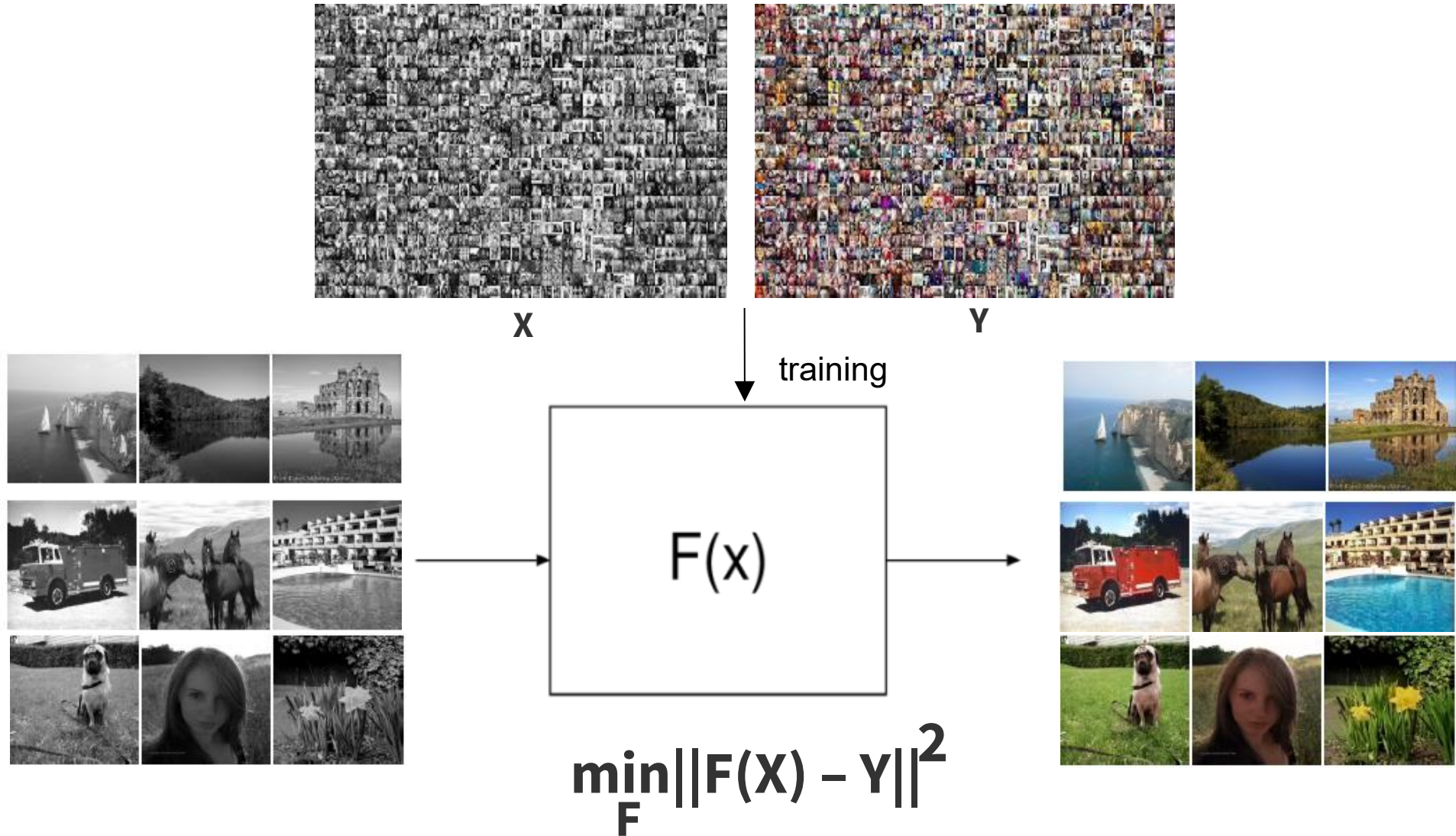
Berry Field, June 1909



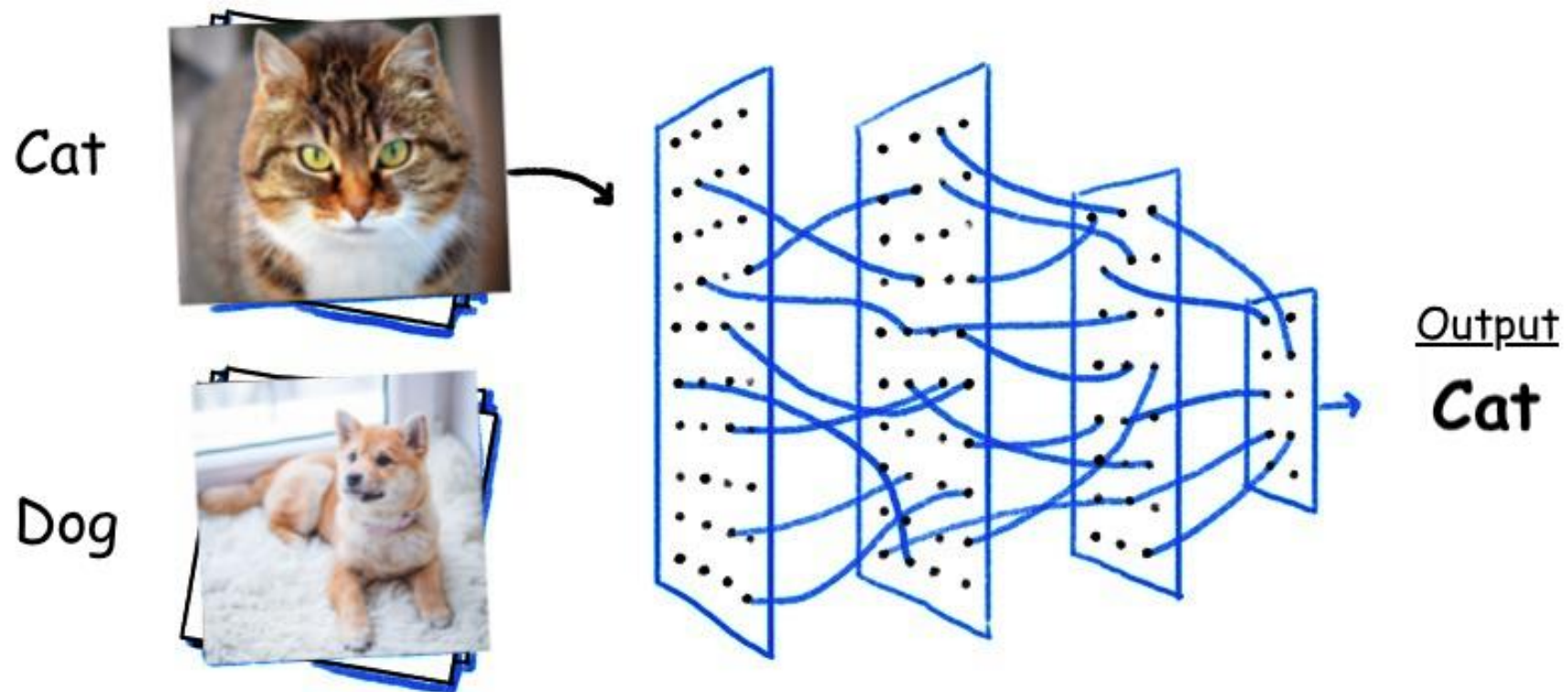
Hamilton, 1936



Supervised Learning



Classification



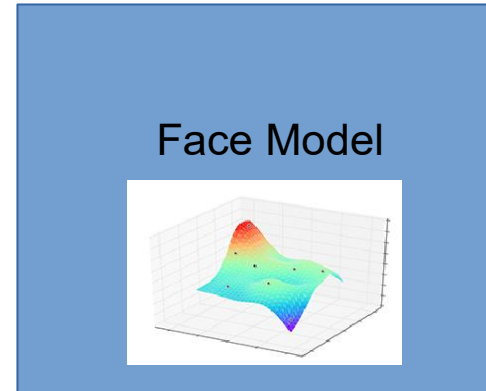
Deep Learning

- ◆ Synthesizing new faces

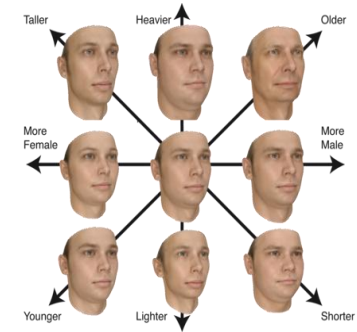
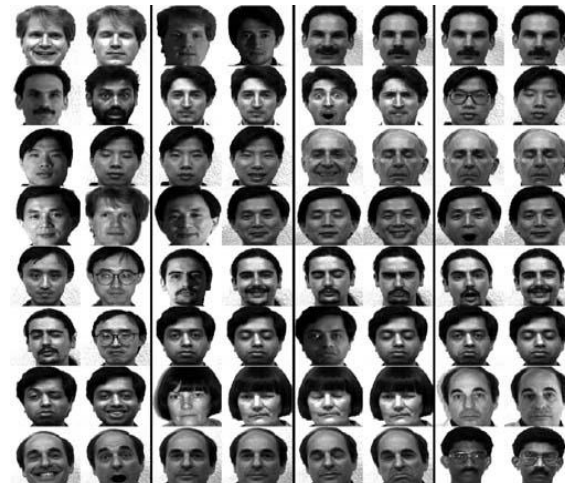


Unsupervised Learning

Learning the space of data



training



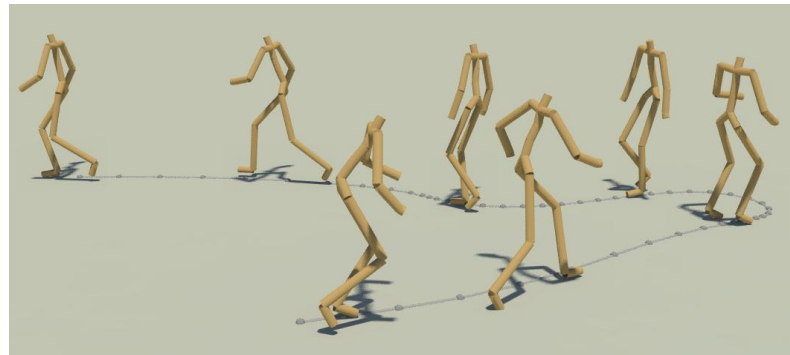
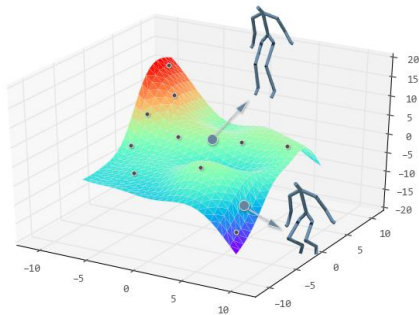
Can design any faces

Deep Learning: Merits

- ◆ Can learn from a lot of data
- ◆ Automatic feature extraction
- ◆ Parallel friendly
- ◆ Can handle nonlinearity
- ◆ High compression rate
- ◆ Fast data reproduction

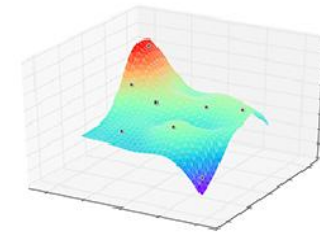
Deep Learning for Character Animation

- ◆ Learn manifold/prior of motion data using *Convolutional Autoencoder*
- ◆ Perform regression from control parameters using *Convolutional Feedforward Network*.
- ◆ Edit generated motion by performing *Optimisation in the Manifold Space*.



Motion Data

- ♦ Motion data is represented as *joint positions*.
- ♦ Valid motion data only exists in a small subspace of this representation.
- ♦ This subspace is called the *Motion Manifold* and is what we wish to construct.



Convolutional Neural Networks

- *Convolutional Neural Networks* (CNNs) convolve over dimensions with local coherence.
- Great successes in Image Classification, Video Classification, Speech Recognition.
- Convolutional neural networks are those where the network weights are learned in the form of convolutional filters that slide over the image.

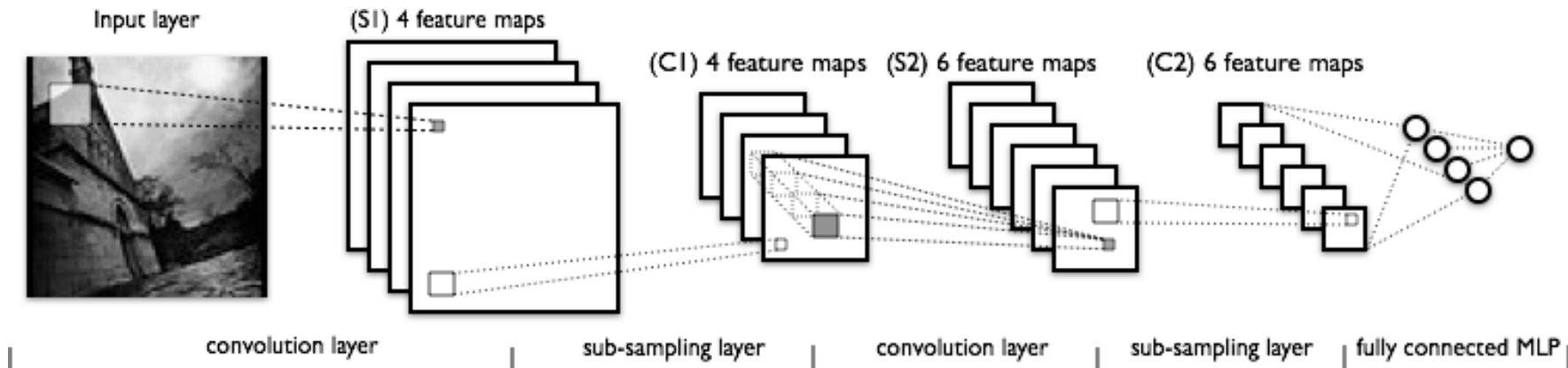
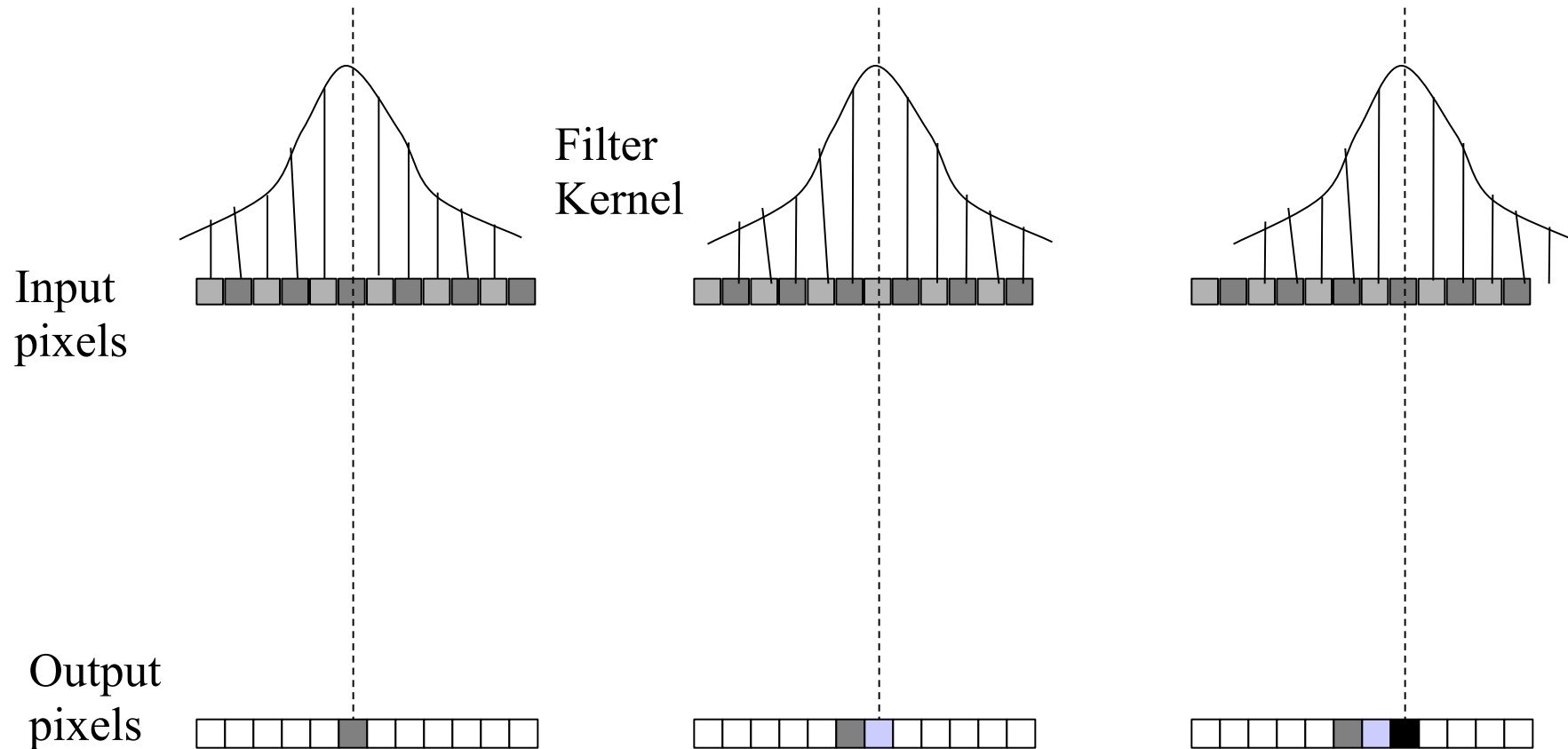
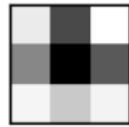
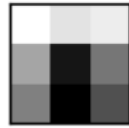
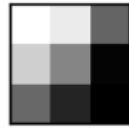


Image convolution – how?



- Each **output pixel p'** is computed as a **weighted sum of pixel neighbourhood of corresponding input pixel p**
 - weighting / size of neighbourhood defined by kernel filter

3x3 filters



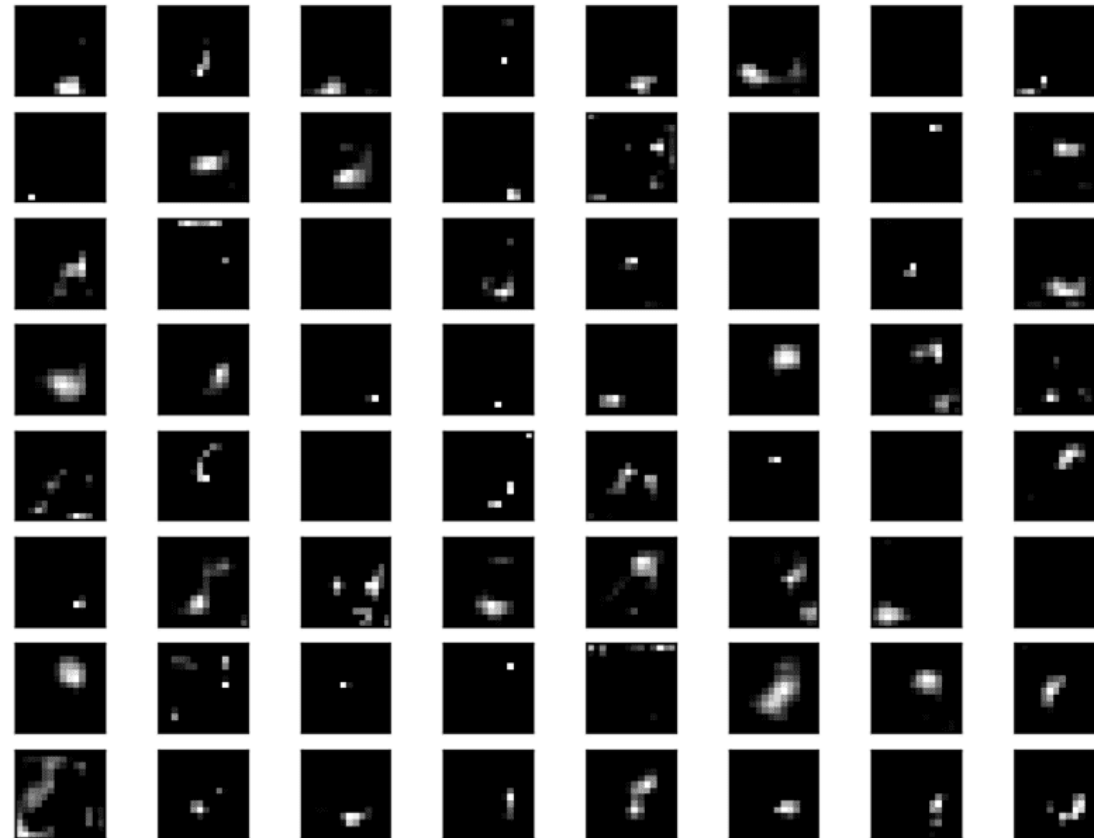
Input image



After applying the filters



Visualizing the feature maps in deeper layers

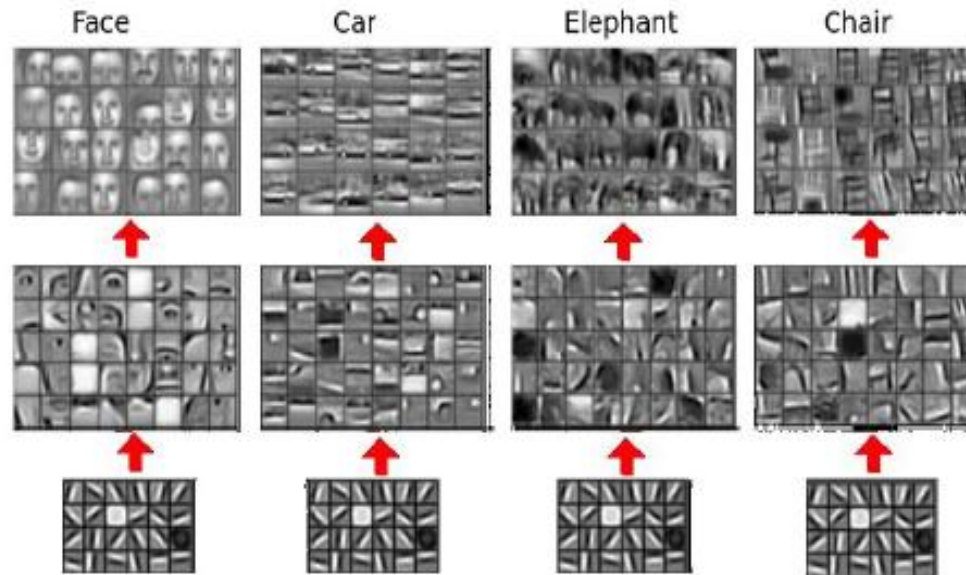


Automatic Feature Extraction by CNN

They can automatically learn features from data which were previously provided by humans

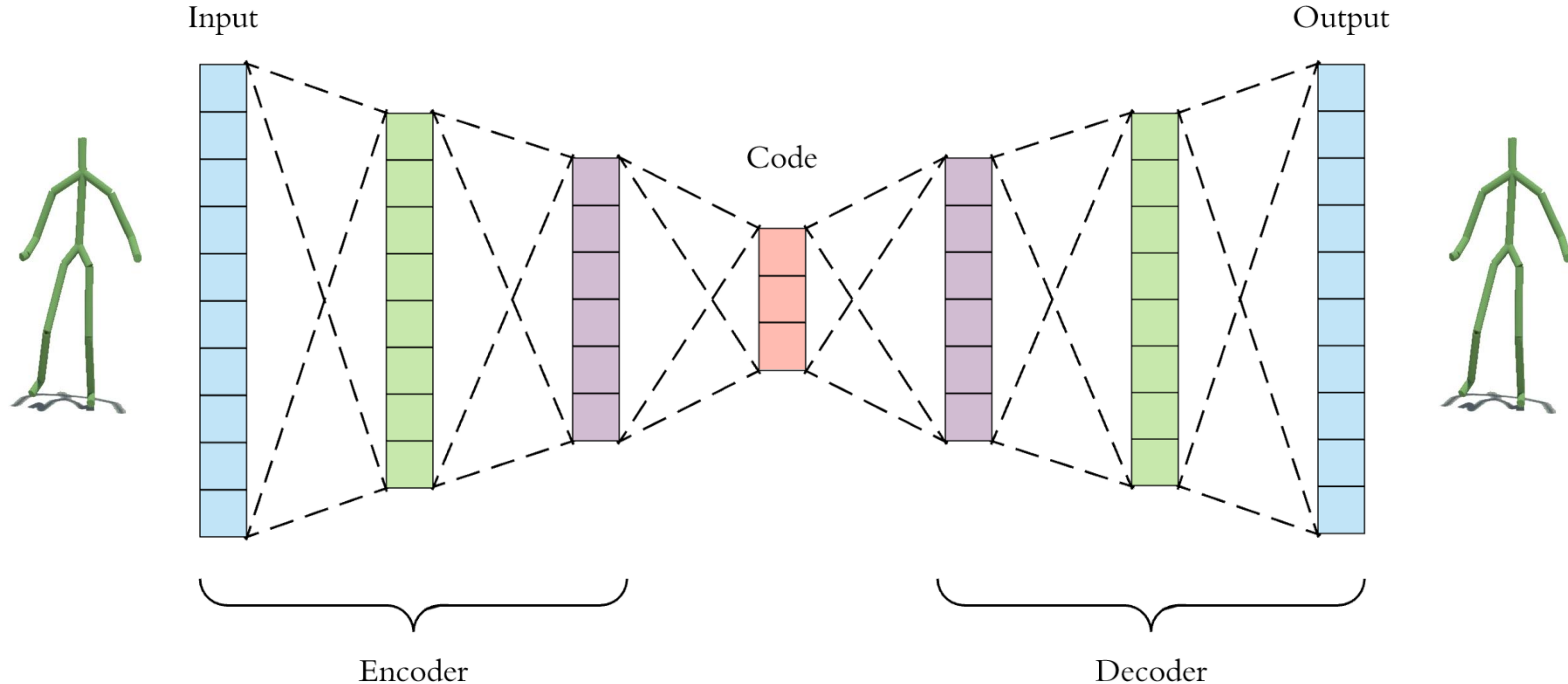
They can learn from low-to-high-level features

We can apply this to Motion Data too.



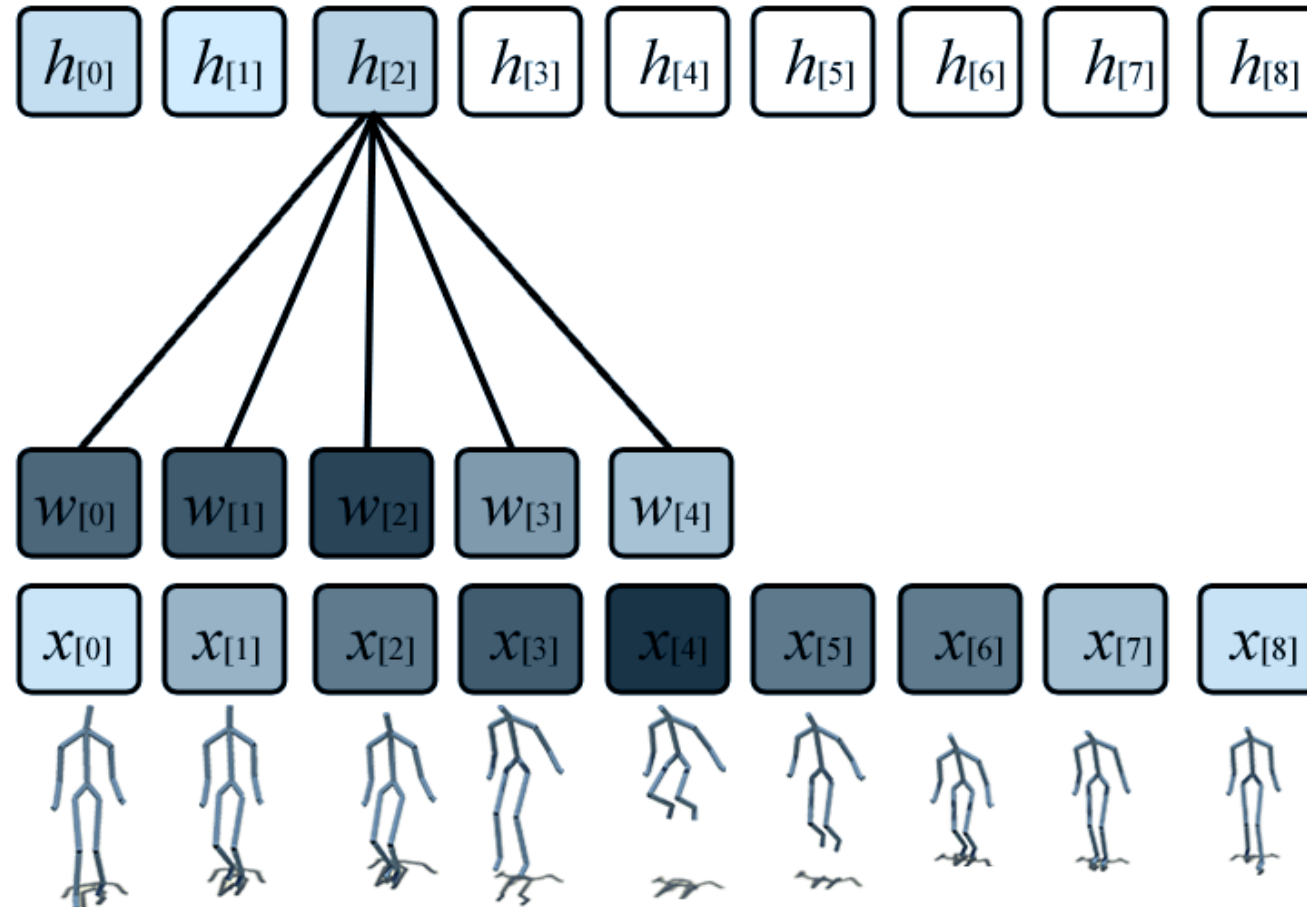
Autoencoder

- Encodes the input data into a latent code
- Composed of the encoder and the decoder
- Trained to reconstruct the original data



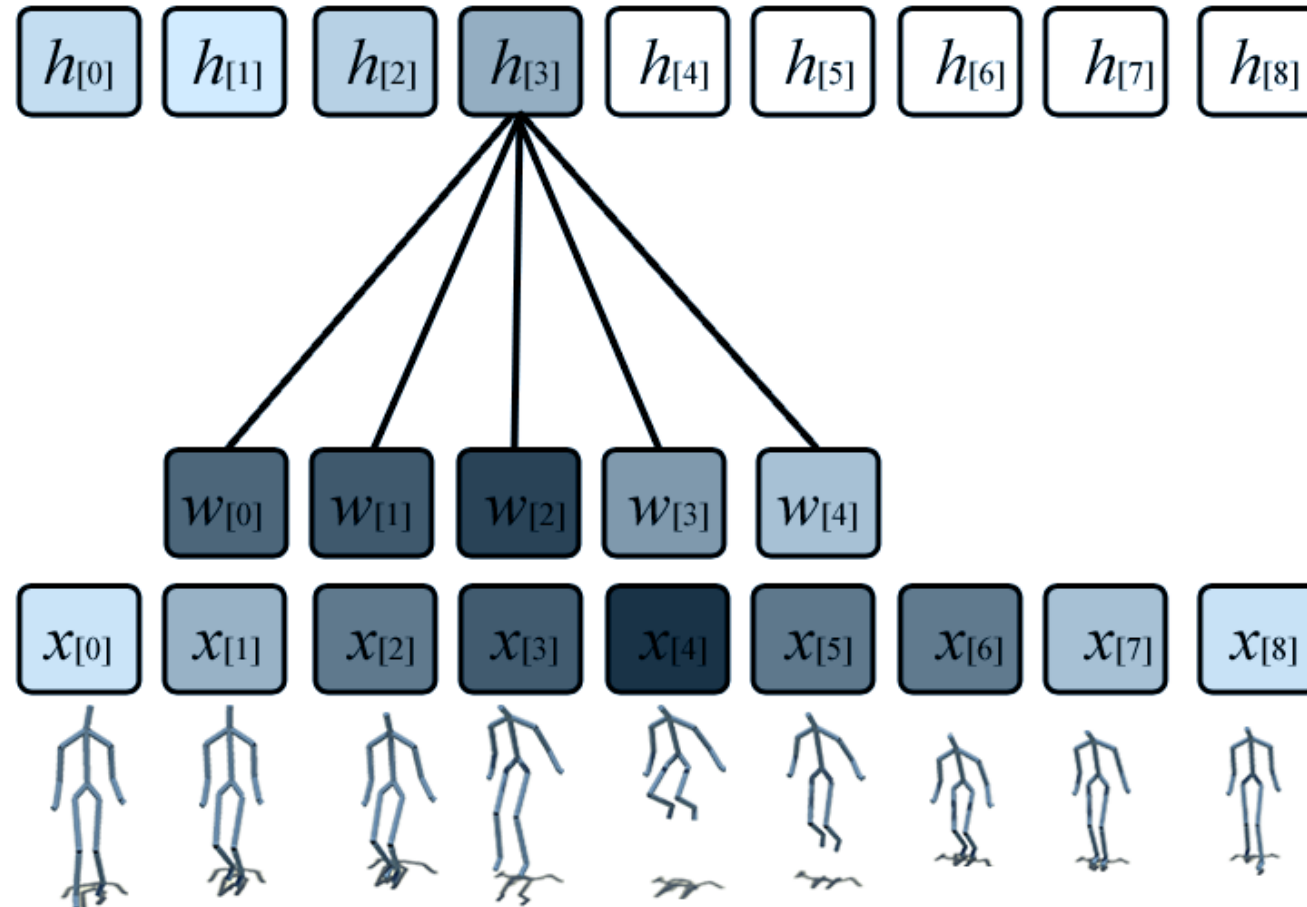
Convolution

- Filters convolve over the temporal dimension.



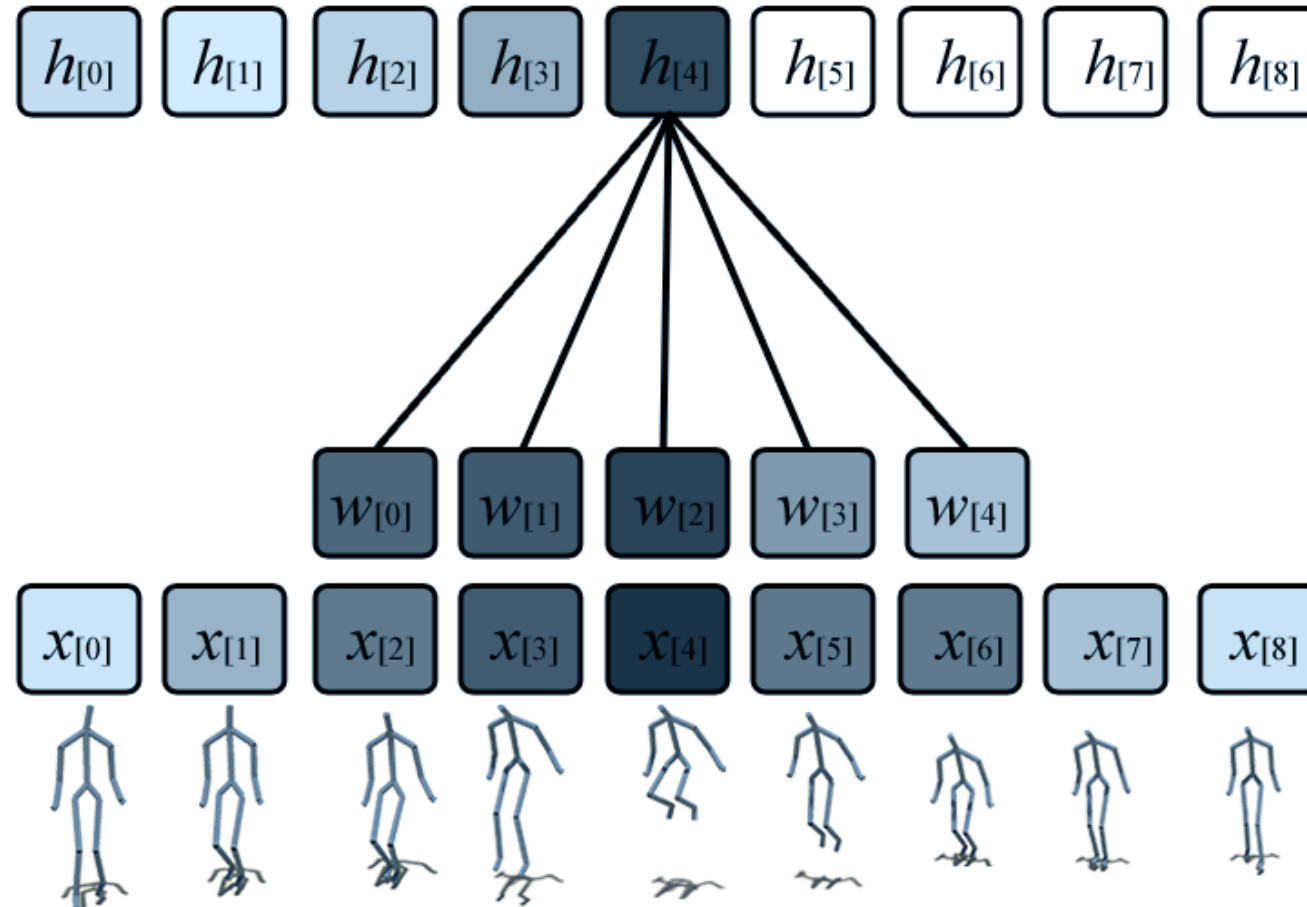
Convolution

- Filters convolve over the temporal dimension.



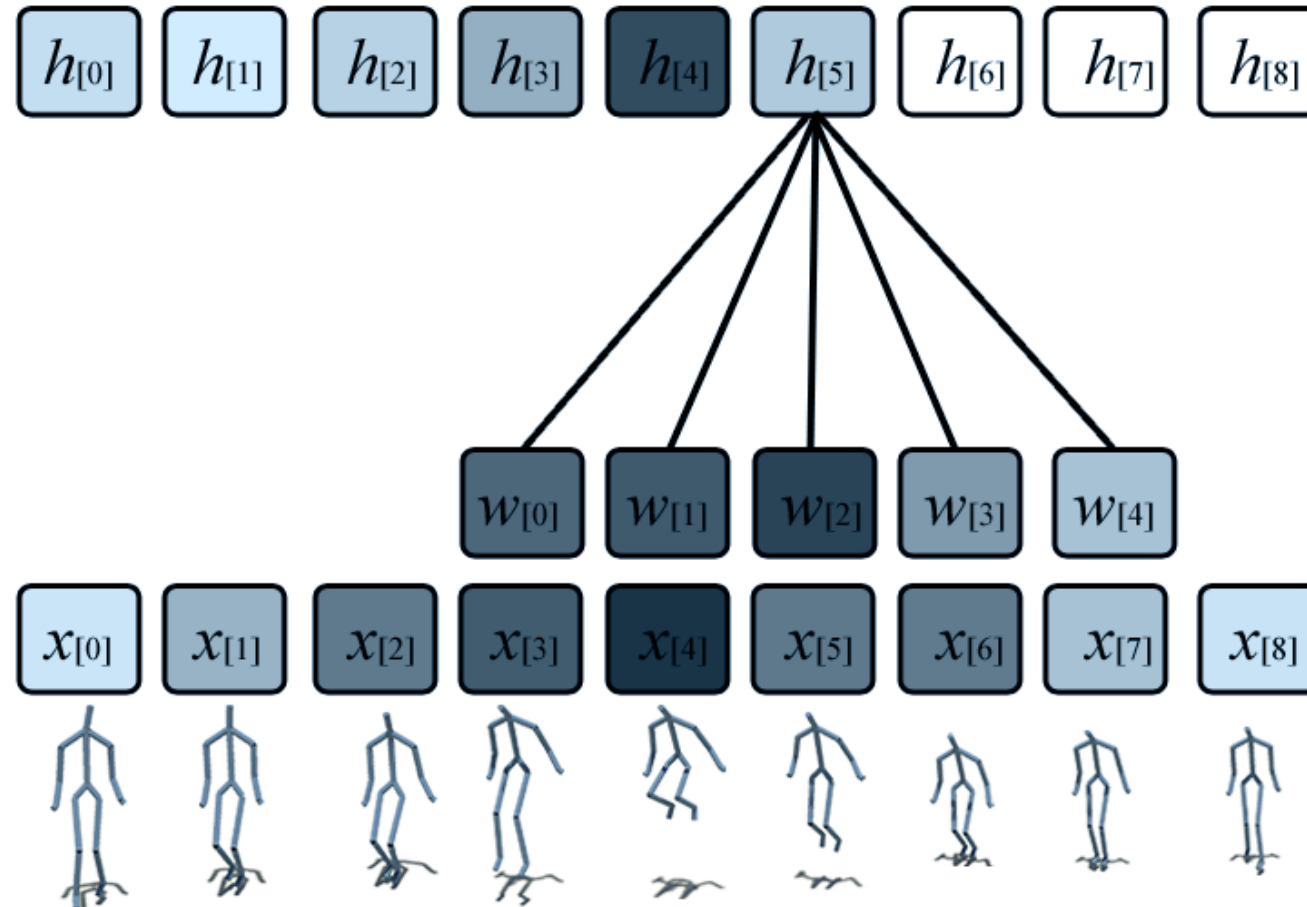
Convolution

- Filters convolve over the temporal dimension.



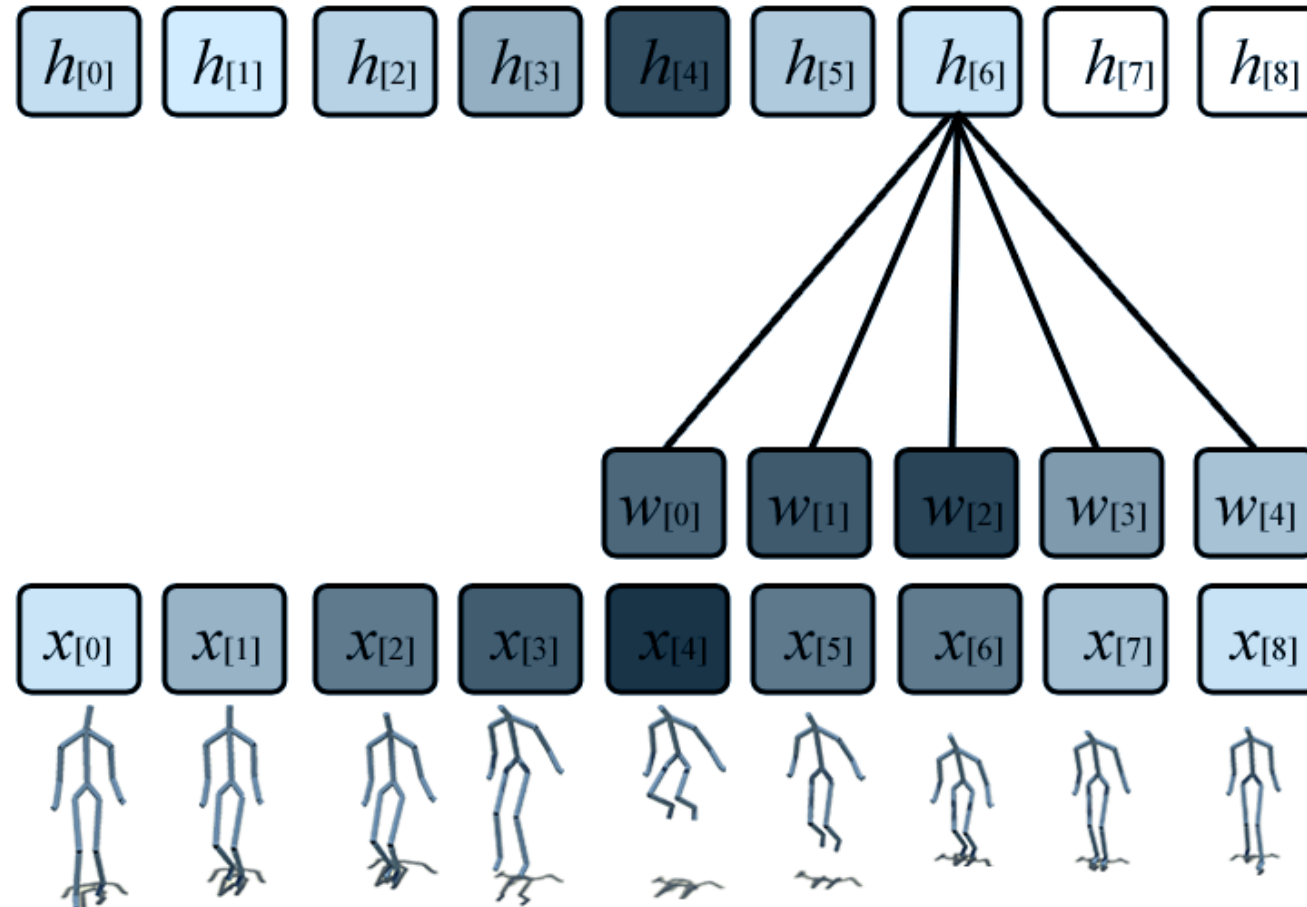
Convolution

- Filters convolve over the temporal dimension.



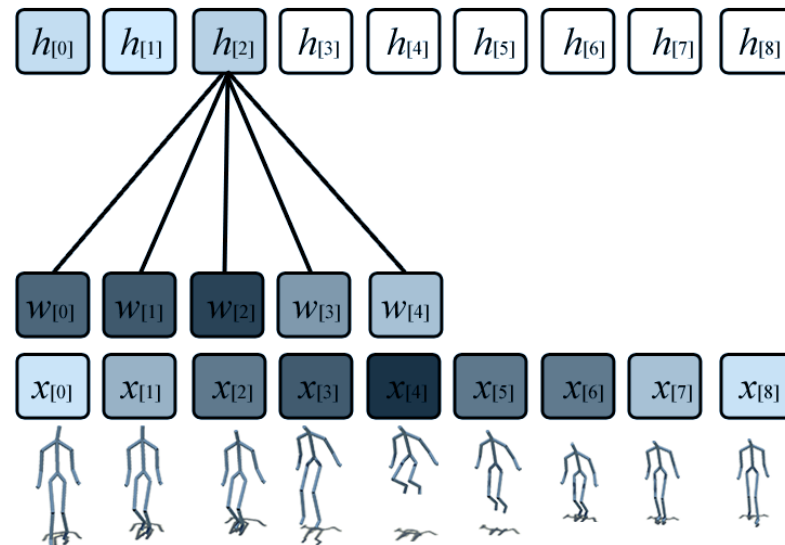
Convolution

- Filters convolve over the temporal dimension.



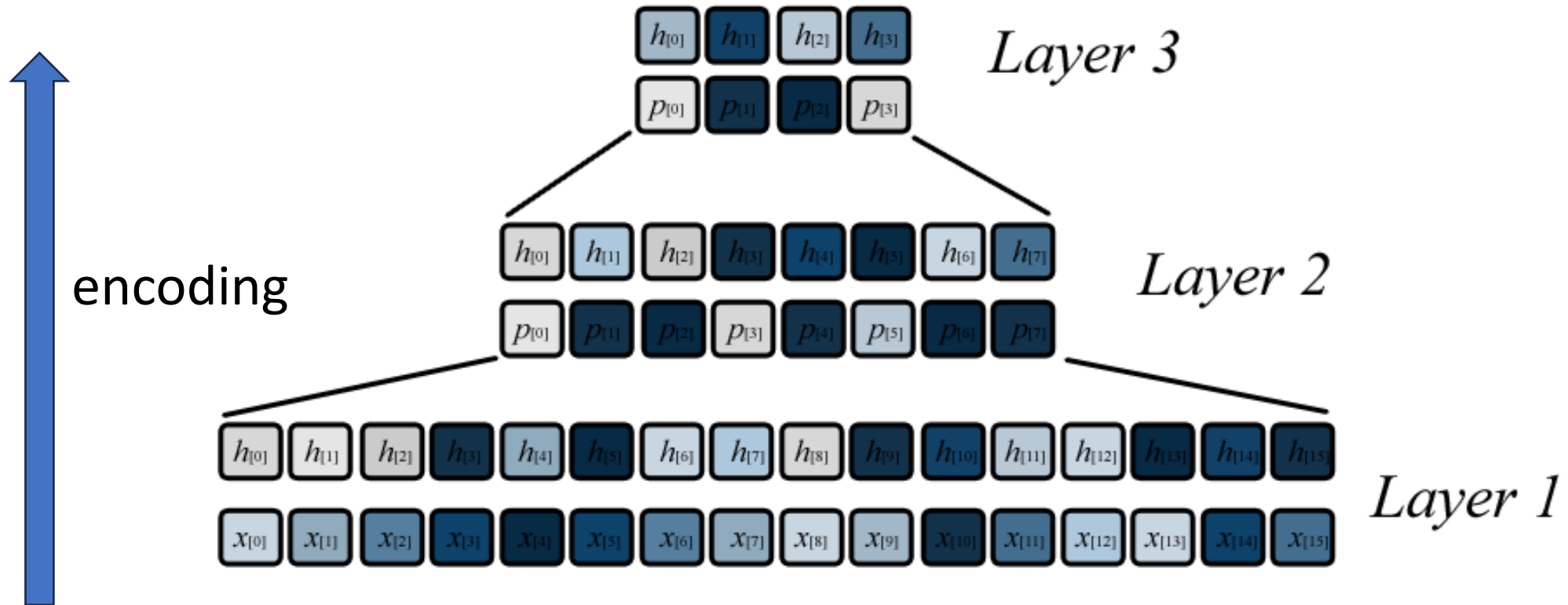
Convolution

- *Hidden Units* are high when the *Filters* match data.
- Each filter covers all degrees of freedom of the body
- Each Filter captures different elements of motion.



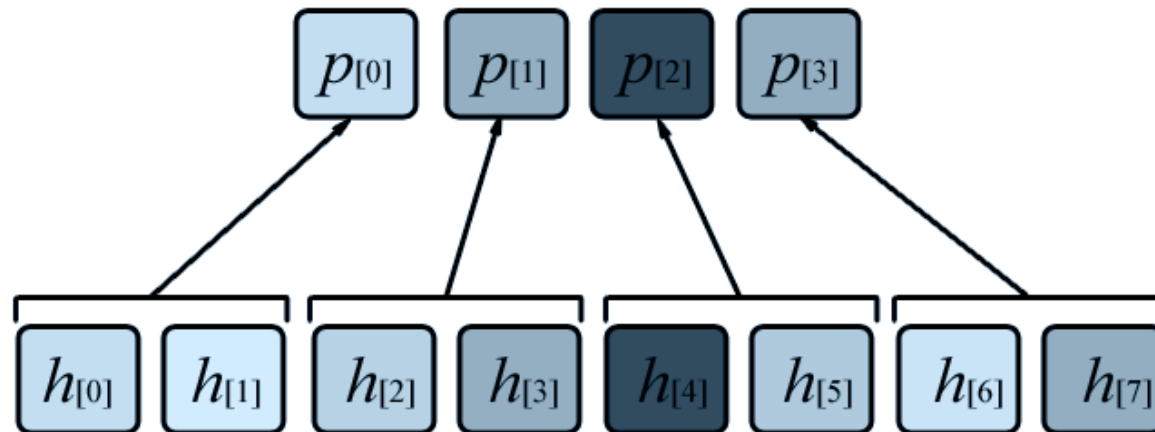
Deep Network

- Whole process is repeated on multiple layers.

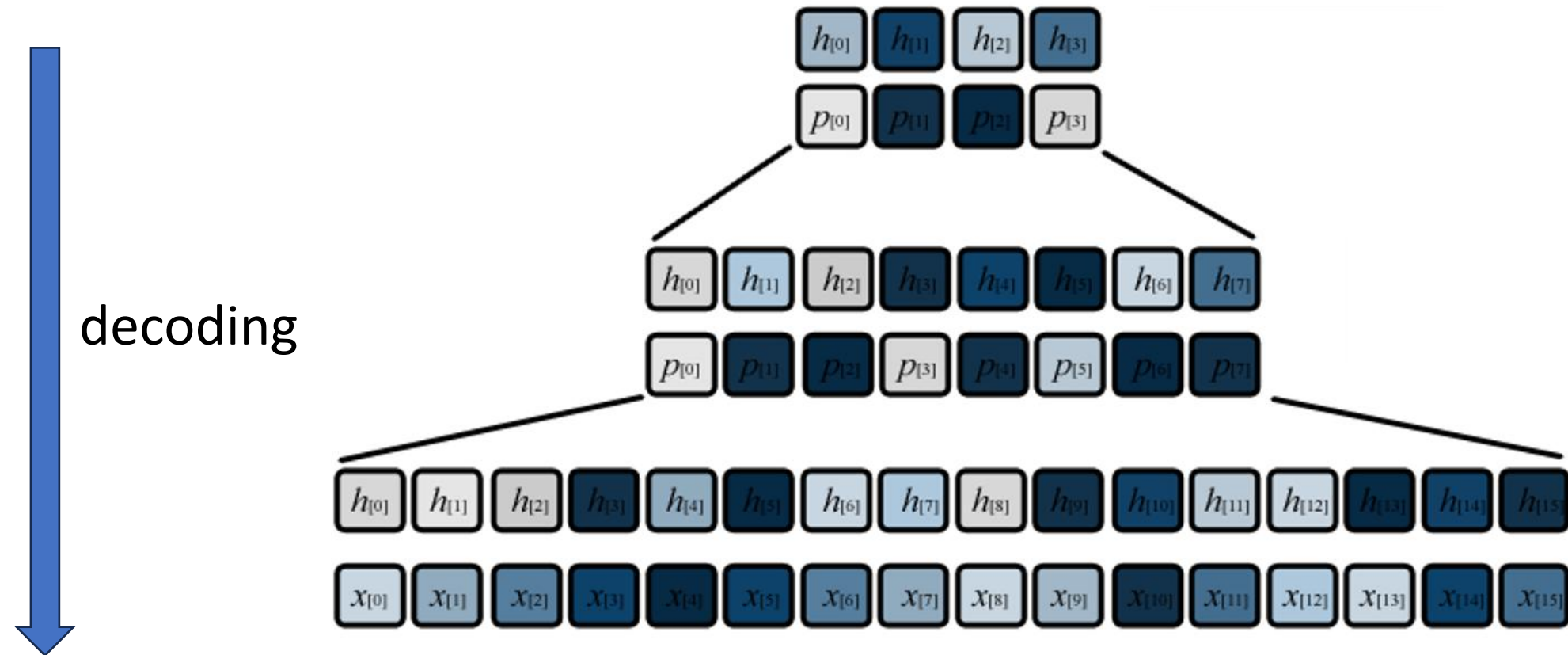


Max Pooling

- ❑ Wish to reduce the dimensionality of the data.
- ❑ *Hidden Unit* values are *Pooled* by taking the *maximum* value over consecutive frames.
- ❑ Forces *Filters* to learn independent features.

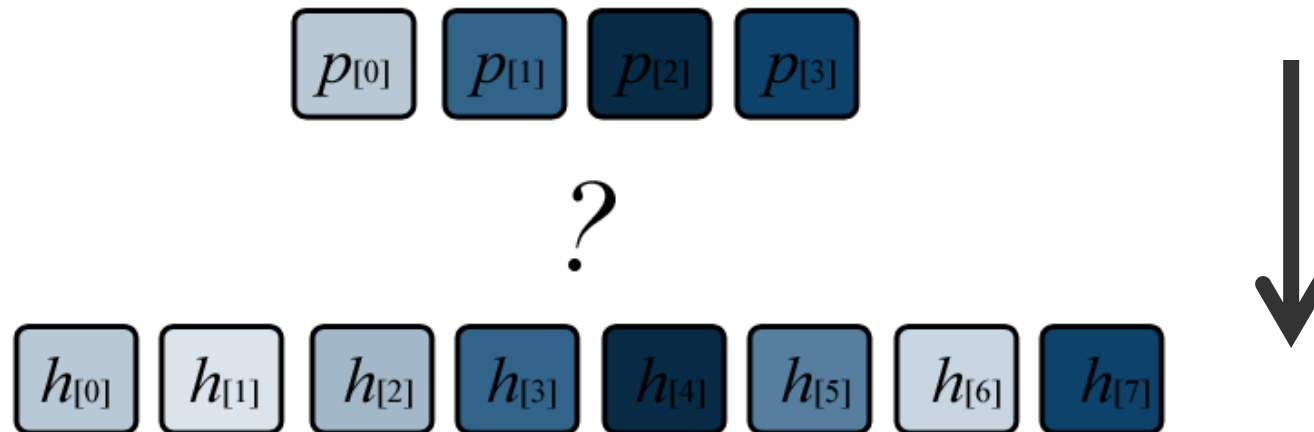


Decoding



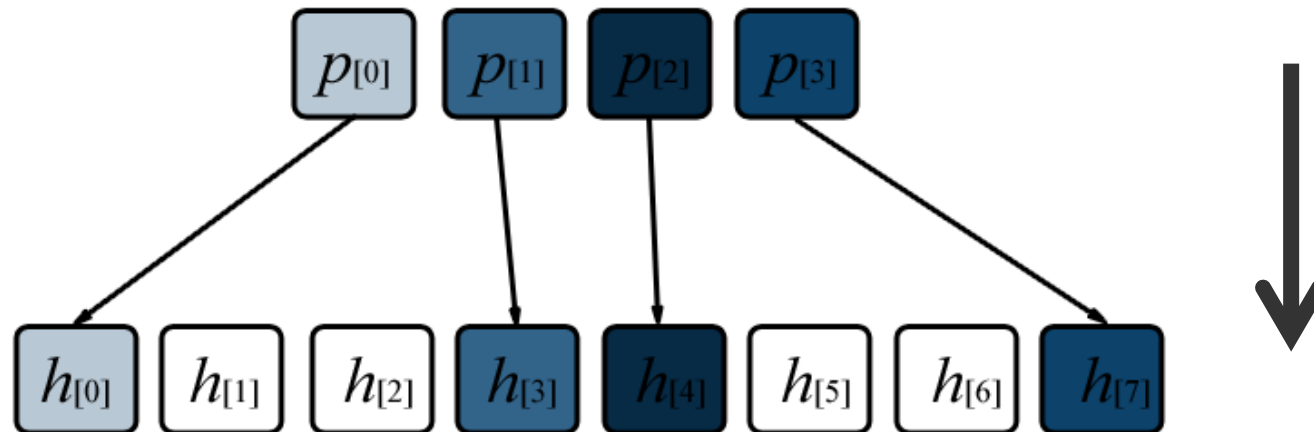
Inverse Pooling

- ▣ Pooled values must also be converted to *Hidden Units*.
- ▣ How to invert this pooling operation?



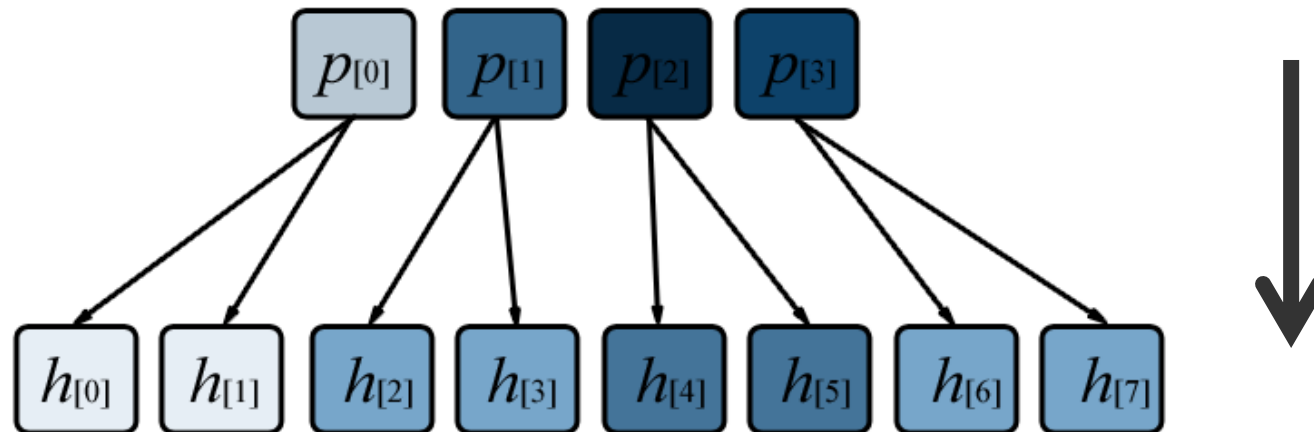
Inverse Pooling

- When *training* randomly pick *Hidden Unit* to place pooled value.
- Approximation of the inverse of the *maximum* operation.



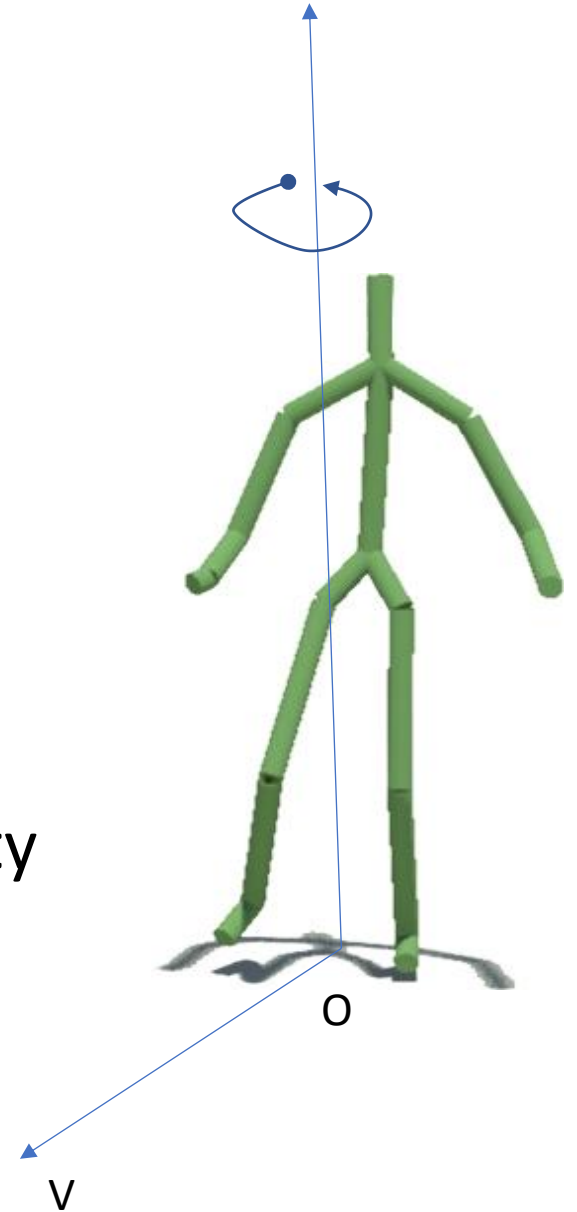
Inverse Pooling

- When doing *synthesis* spread pooled value across *Hidden Units*.
- Produces smoother motion with no noise.



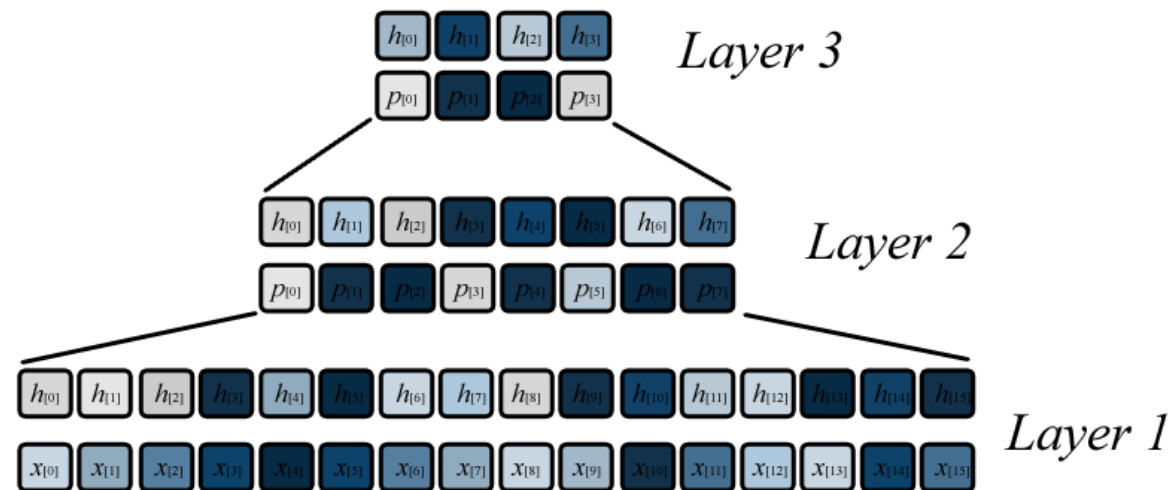
Training

- Full CMU database is used (~10 hours of data).
- Separated into overlapping windows of 160 frames and sub-sampled to 30fps.
- Using Joint Positions relative to the root joint.
- Global translation on XZ plane and global rotation on Y axis are removed.
- Rotational velocity around Y axis and translational velocity on XZ plane (relative to forward direction) are added.



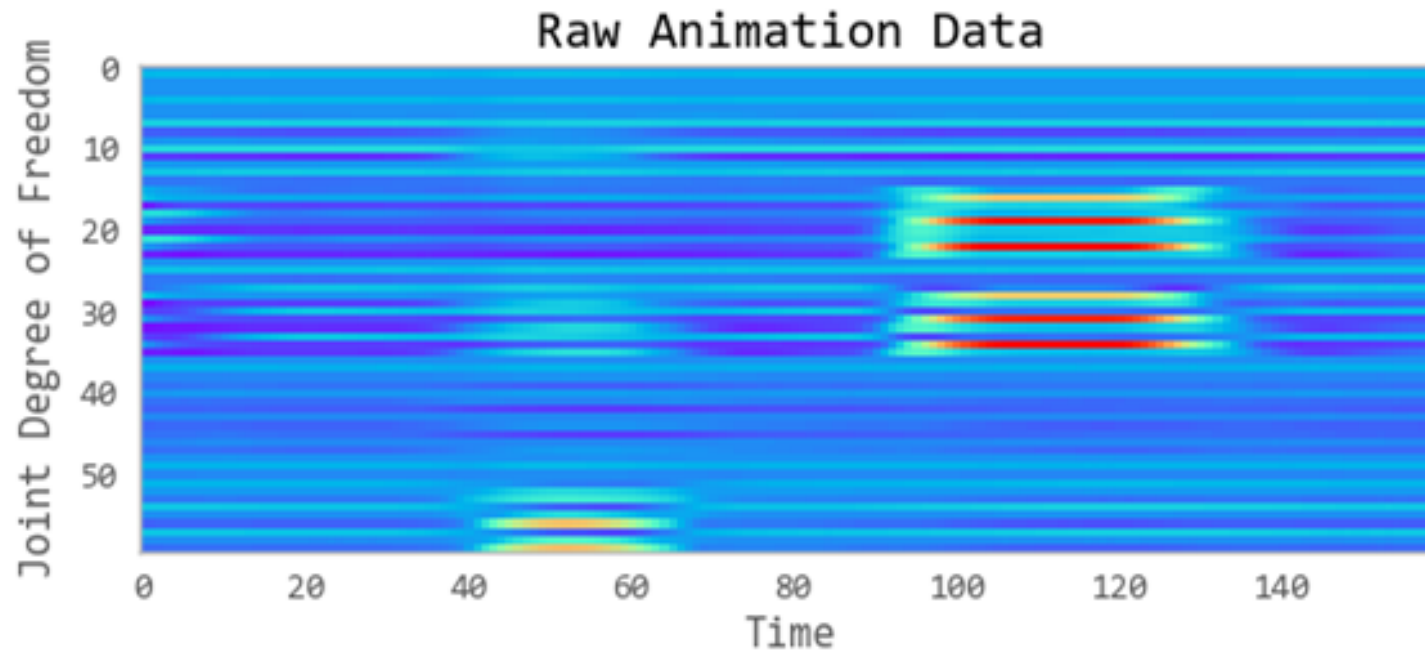
Training

- Three layers
- First each layer is trained independently to reproduce layer below accurately.
- Finally all layers are trained together in a stage called “Fine Tuning”.



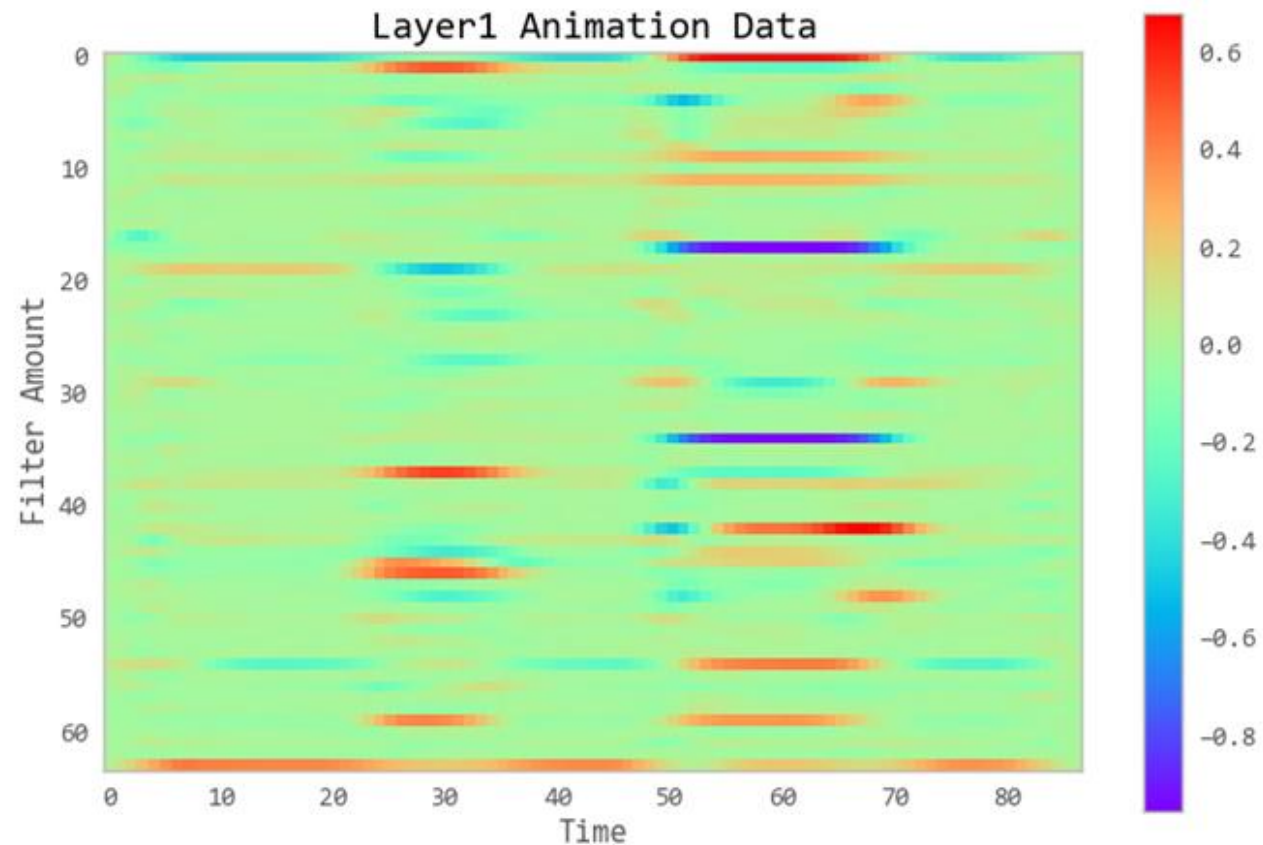
Raw Motion Data

- A person lifting their arms looks like this...



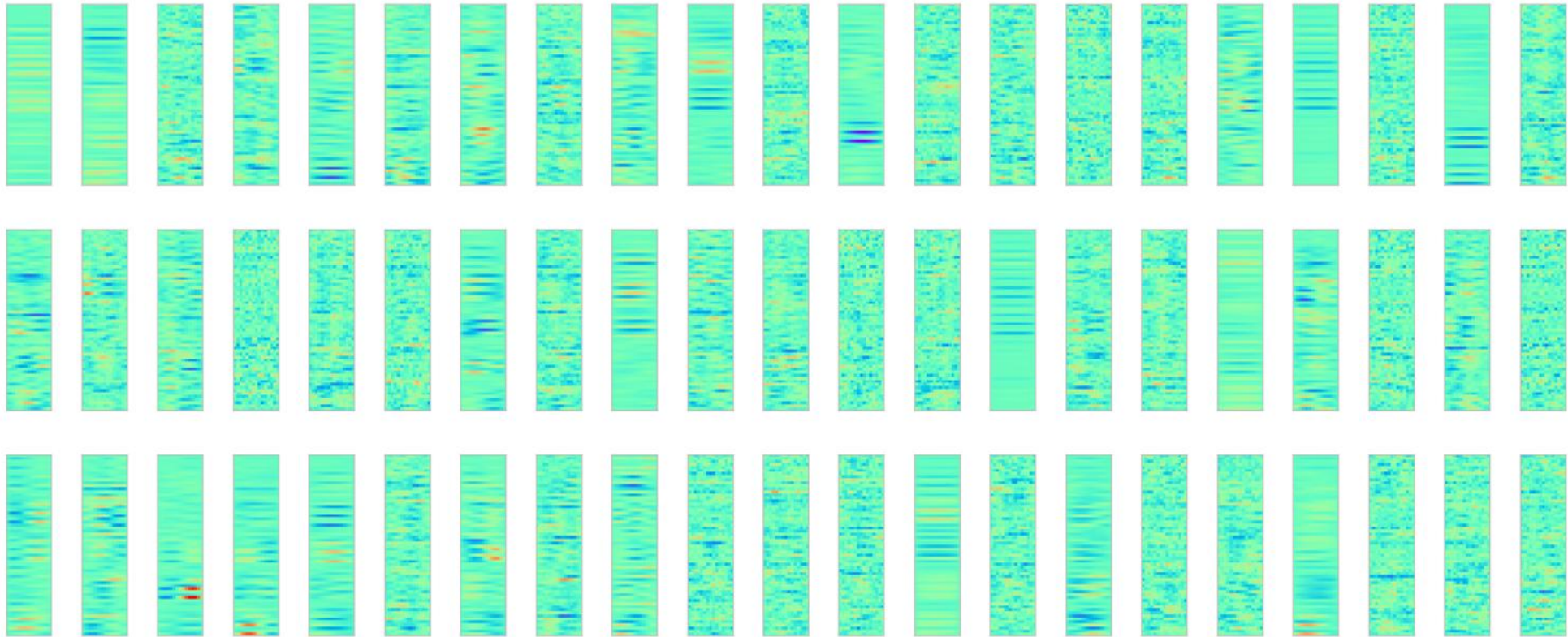
Manifold Motion Data

- After one level of projection features are sparser, clearer...

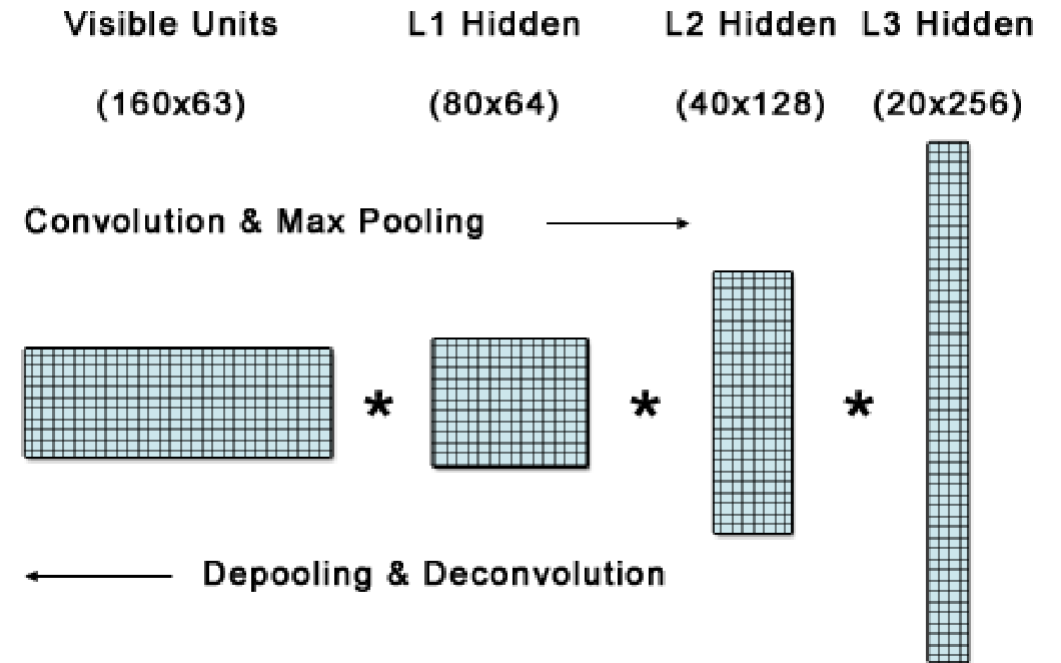
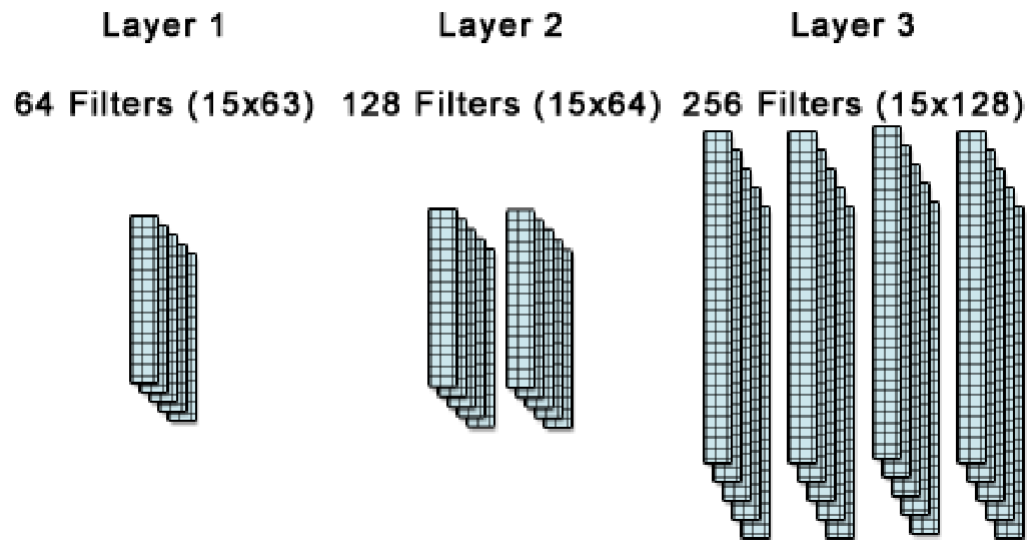


First Layer Filters

- Many strong correlations can be seen in the filters.
- High values across multiple frames and joints.



Filter size and input/output sizes



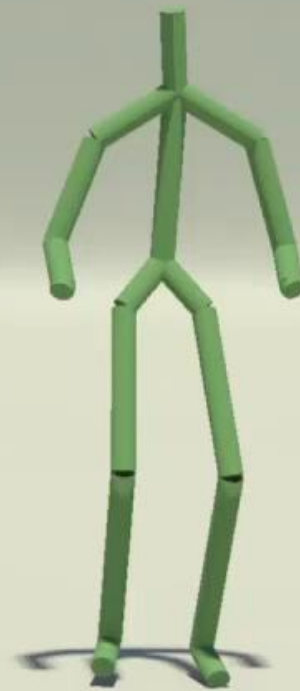
Layer 3 Filters



Filter 5

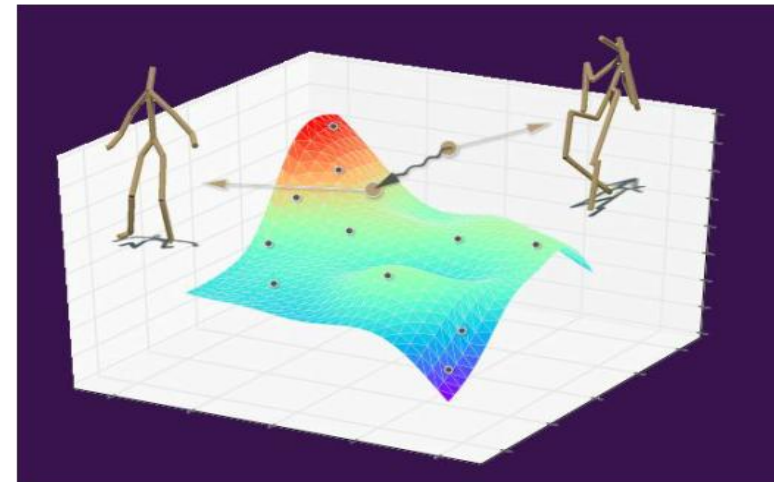


Filter 11



Filter 27

Application



- Motion Denoising
 - Projecting noisy motion onto the motion manifold (latent space) and decode to produce denoised motion
- Motion Interpolation
 - Interpolating the motions in the latent space
- Motion search
 - Lookup for similar motions by comparing the motion in the latent space



Corrupted

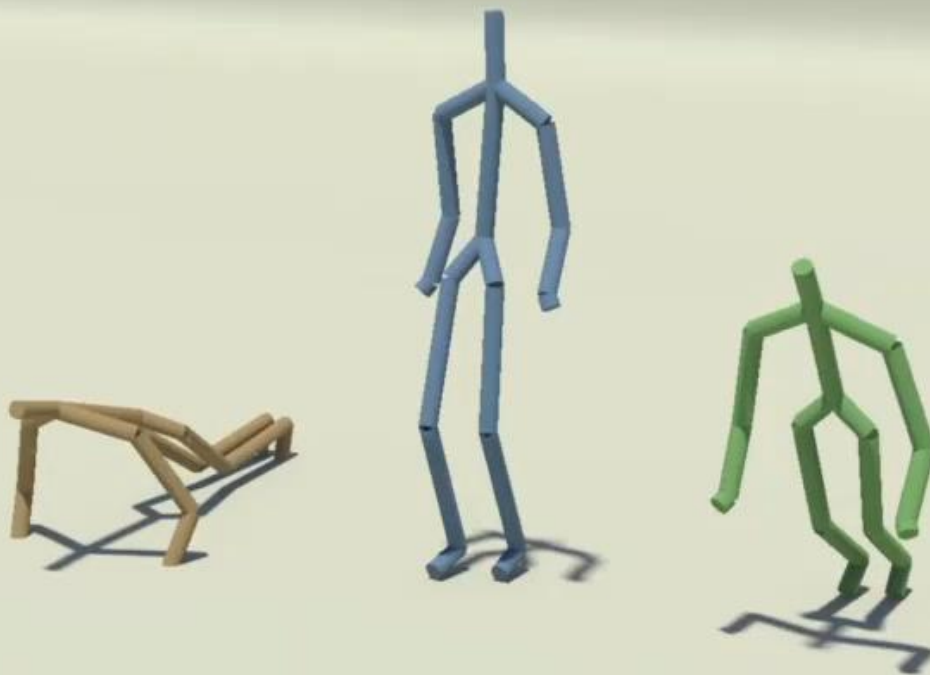


Projected



Ground Truth

Naive Interpolation

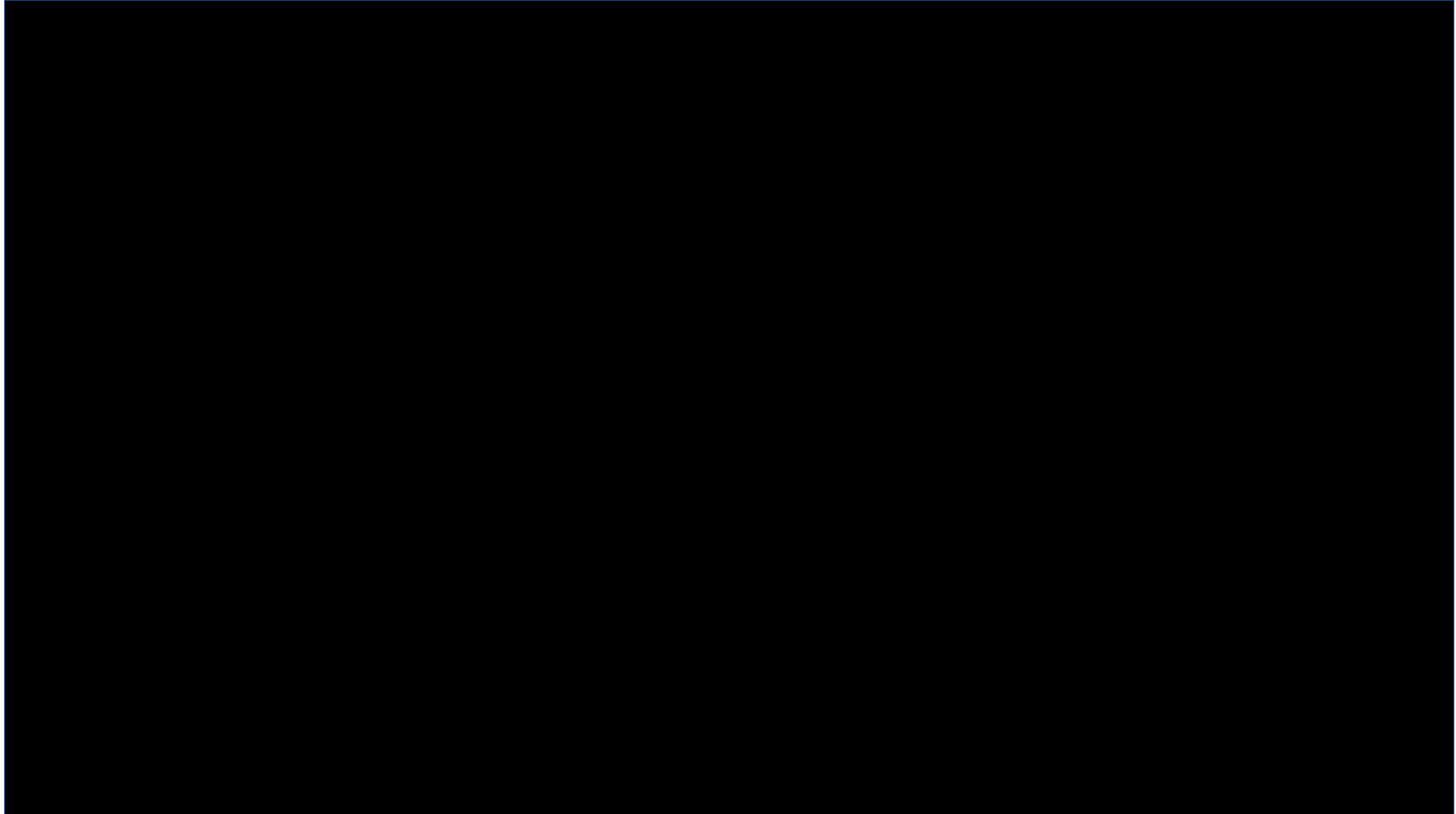


Pushup

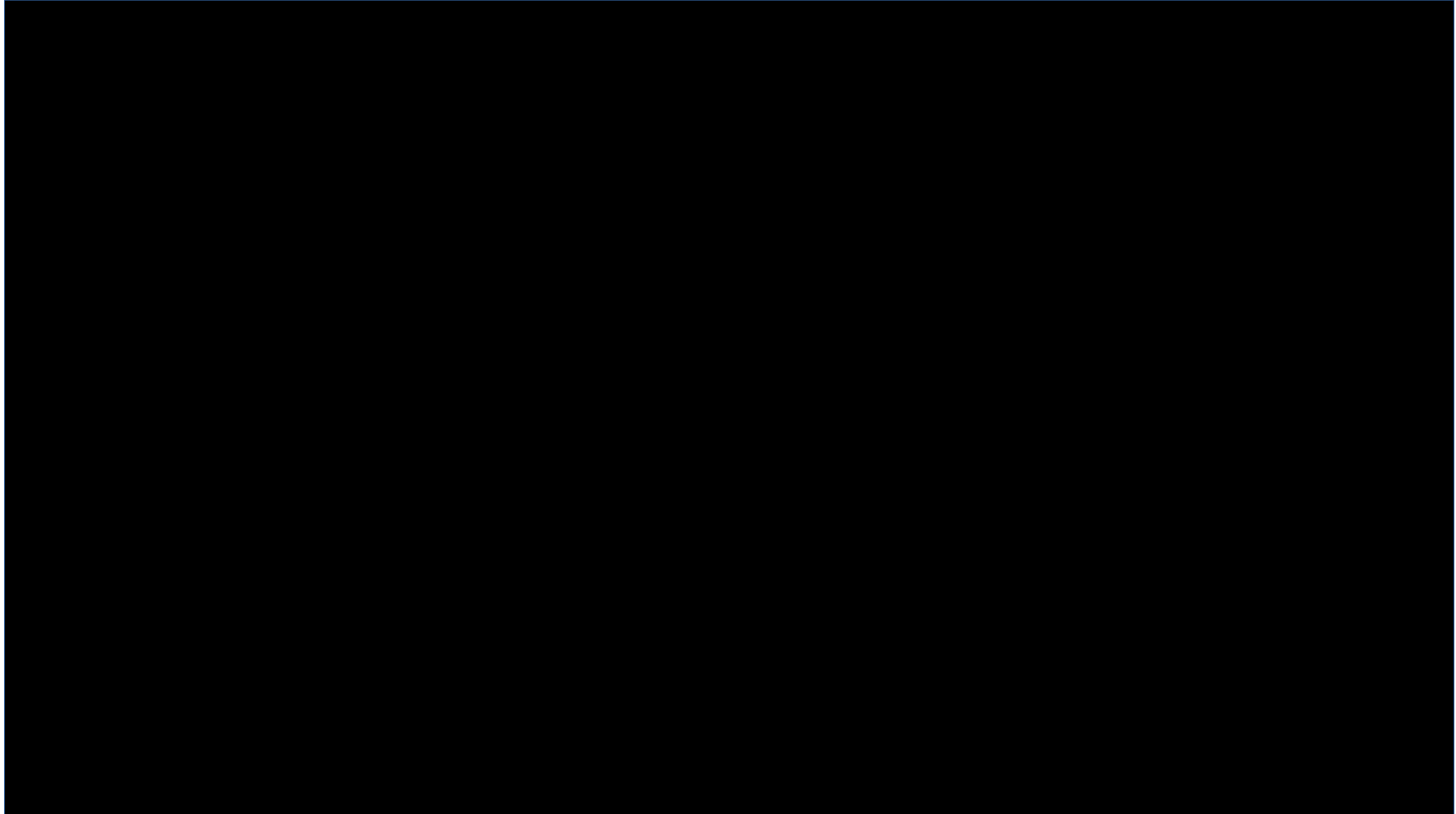
Walking

Interpolation

Similar Motions (Naive)



Similar Motions (Ours)



Discussion

- *A Motion Manifold* can be effectively learned using a *Convolutional Autoencoder*.
- This method scales well to large amounts of data including the whole CMU motion capture database.
- This representation is useful for many different applications in character animation
- The learned manifold is robust because it uses many joint positions across many frames to predict a single *Hidden Unit*.
- Using a Convolution to encoding the temporal aspect of the manifold makes this possible and ensures temporal invariance.

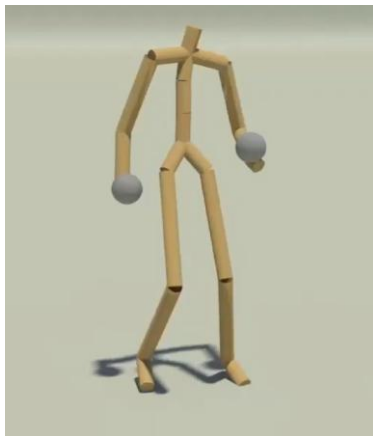
Let's use this for motion synthesis

- We were only using this for searching/denoising/interpolating motion
- How can we use this representation for motion synthesis?
- How to regress the control signals to the full body motion?
- We need a user input - what kind of control signals will be good?



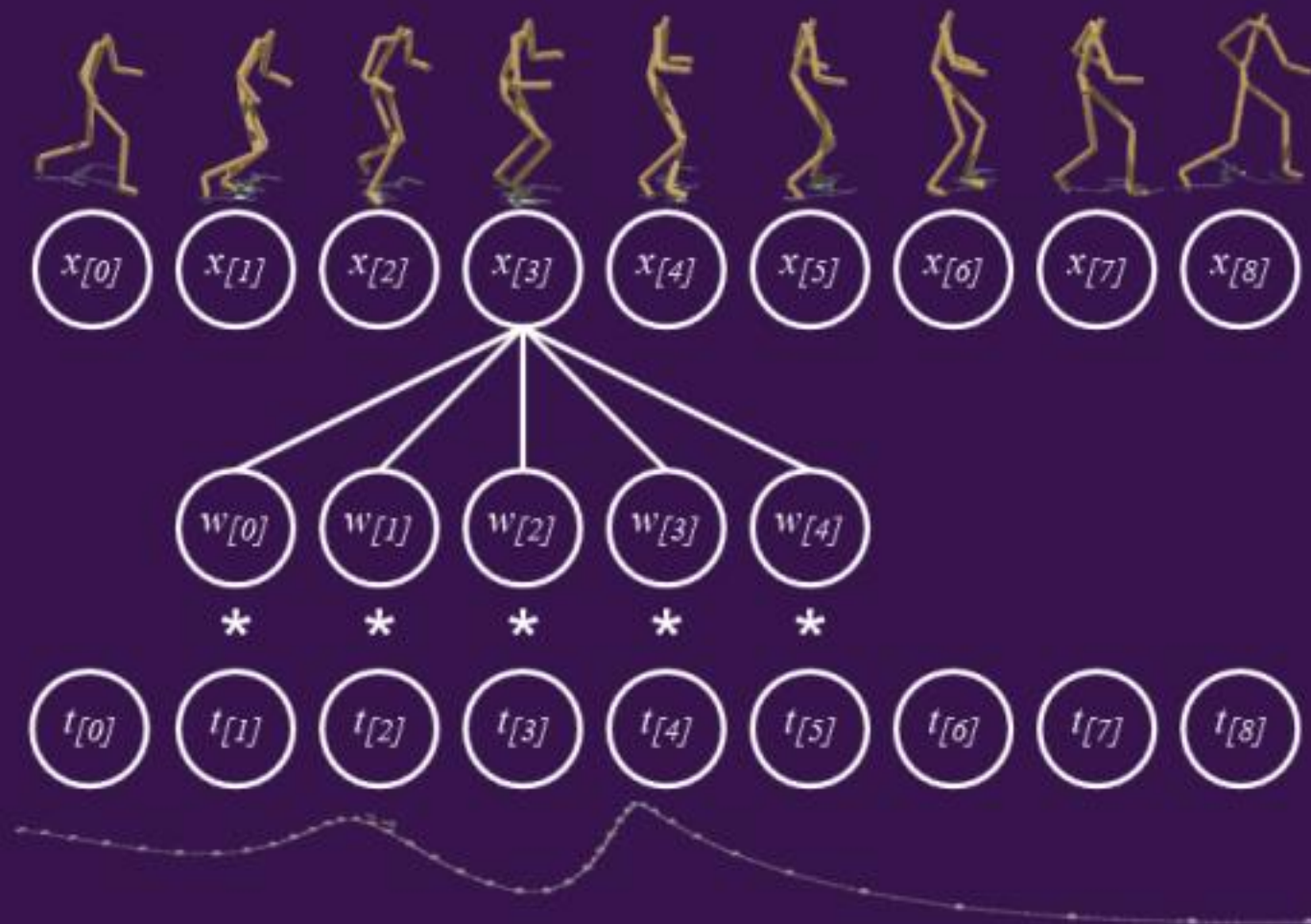
Let's use this for motion synthesis

- We were only using this for searching/denoising/interpolating motion
- How can we use this representation for motion synthesis?
- How to regress the control signals to the full body motion?
- We need a user input - what kind of control signals will be good?



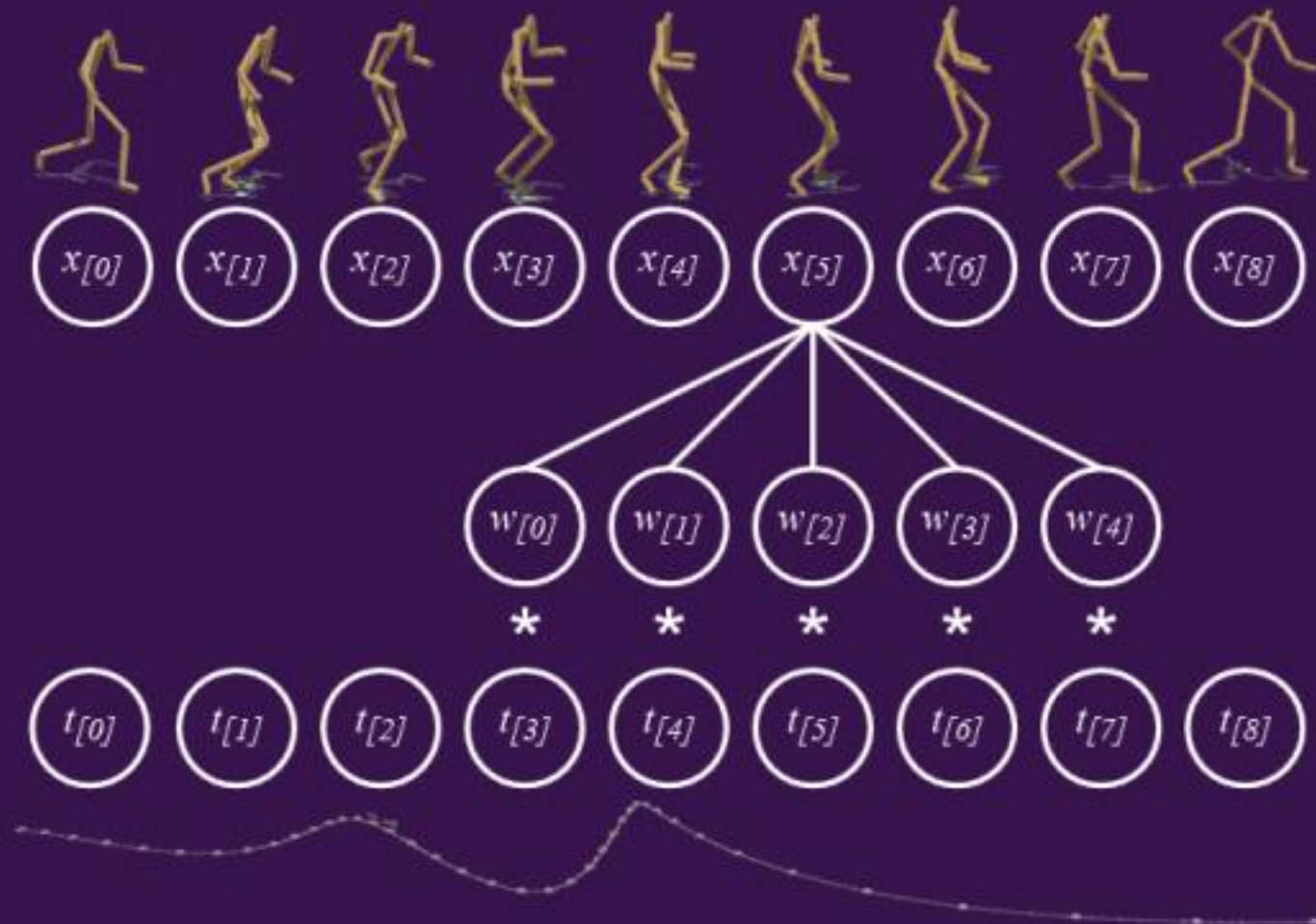
Convolution

Filters convolve over temporal dimension



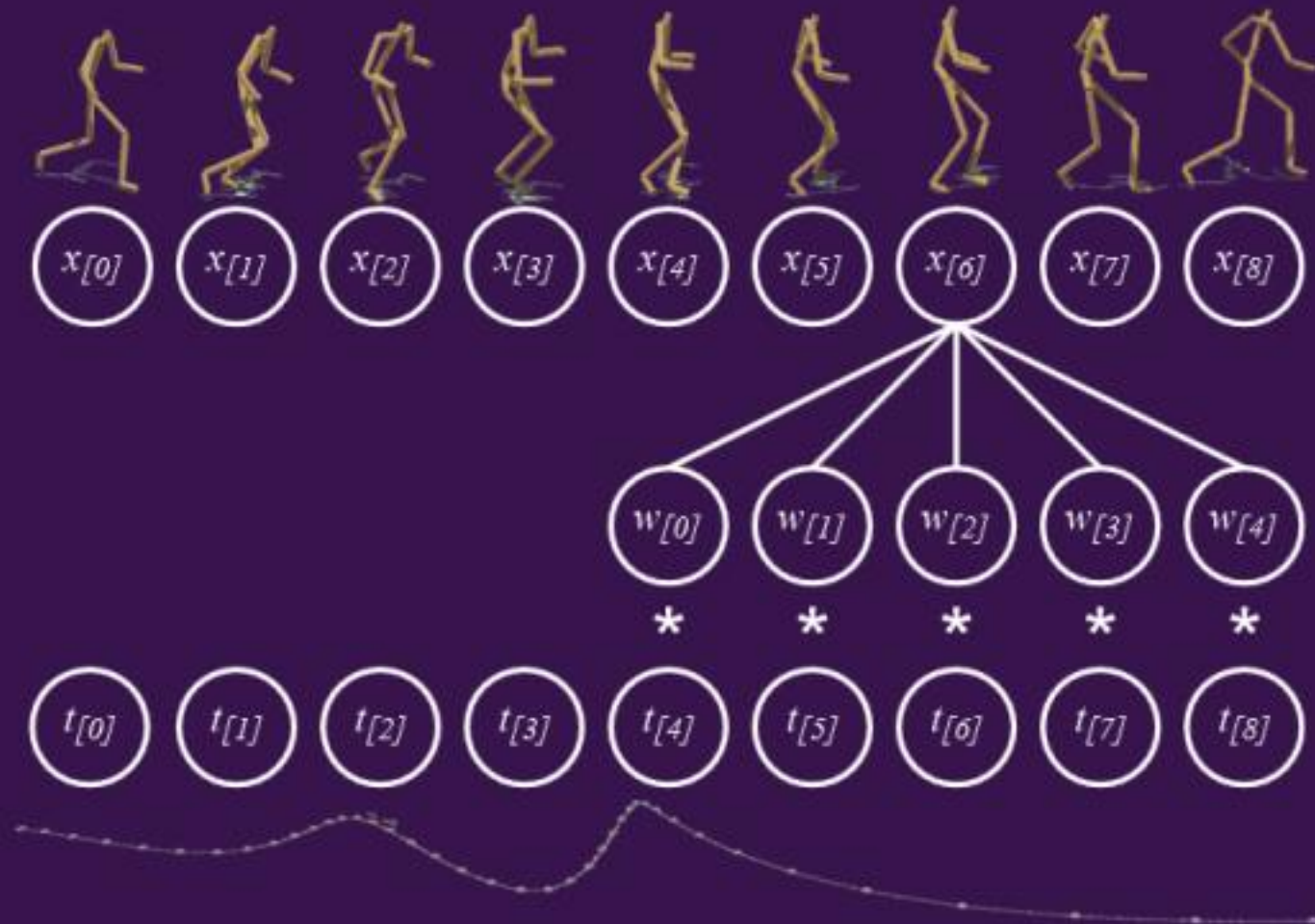
Convolution

Filters convolve over temporal dimension



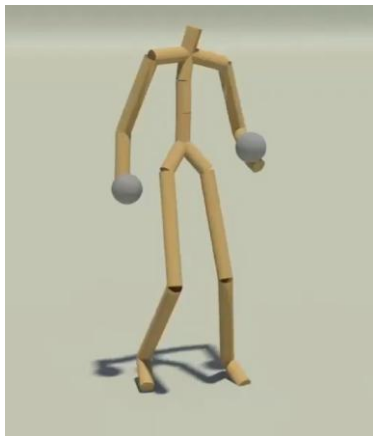
Convolution

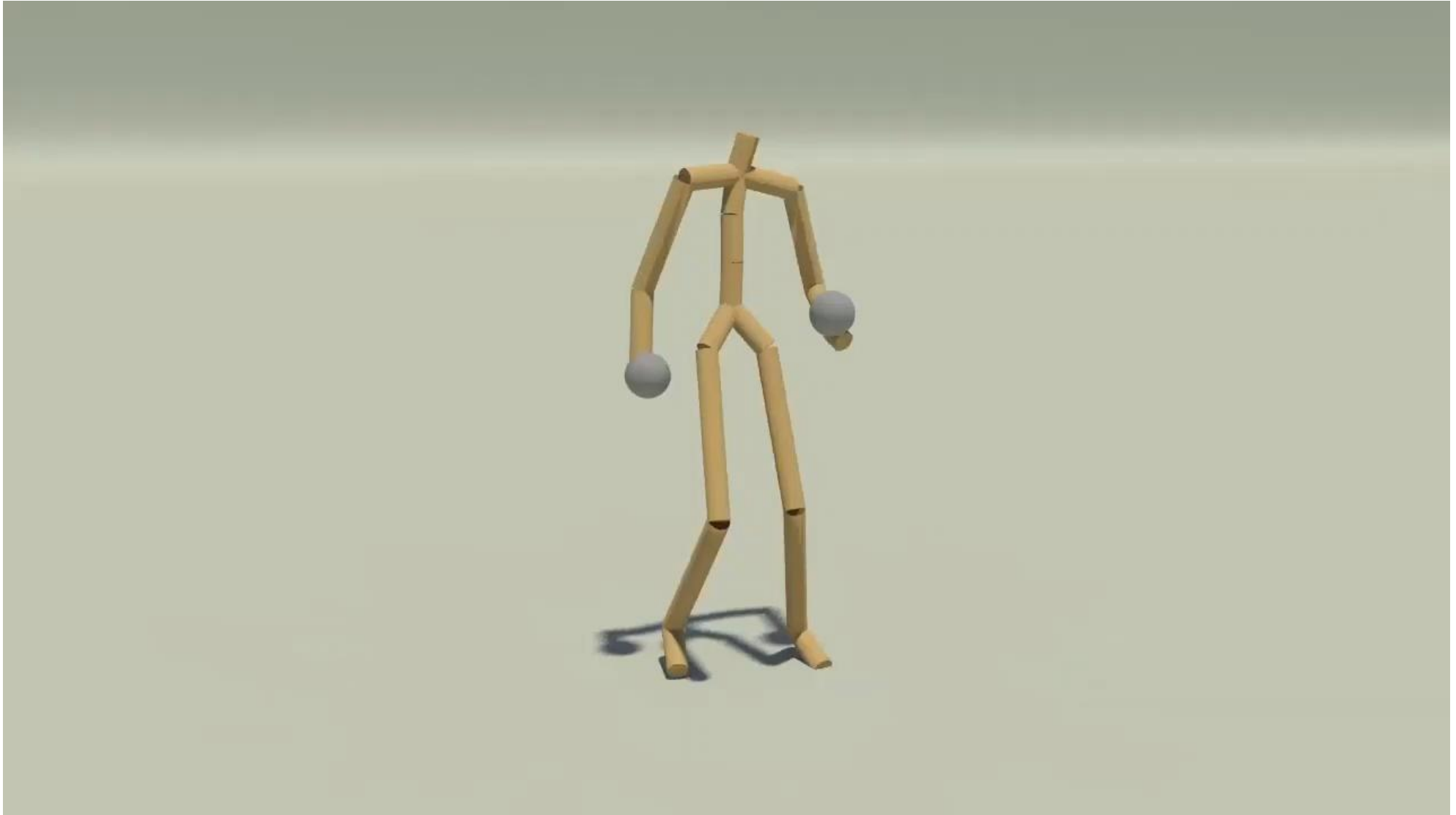
Filters convolve over temporal dimension



Let's use this for motion synthesis

- We were only using this for searching/denoising/interpolating motion
- How can we use this representation for motion synthesis?
- How to regress the control signals to the full body motion?
- We need a user input - what kind of control signals will be good?





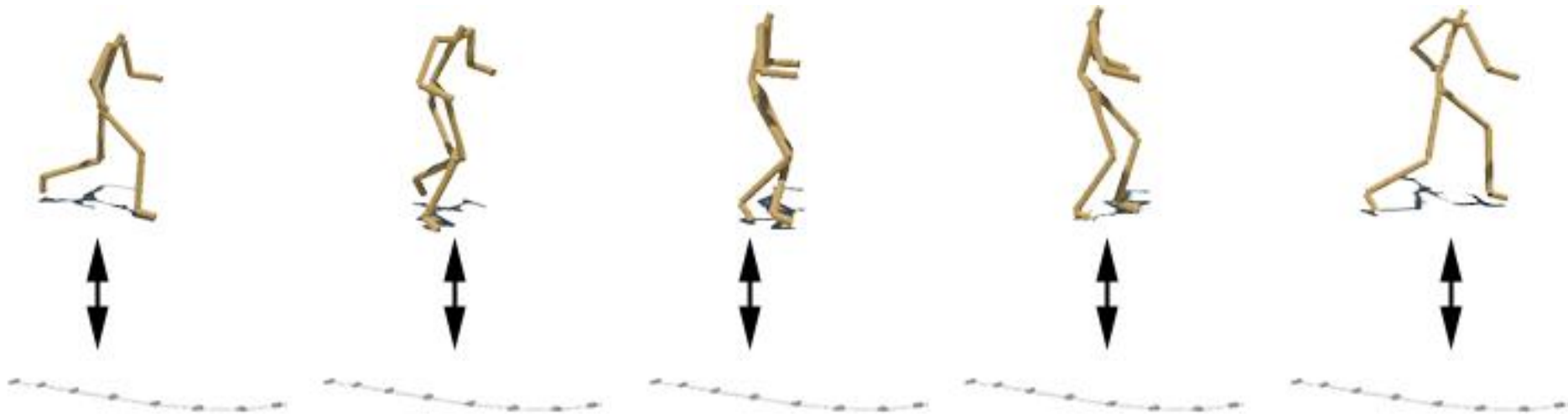
Yes, works well!



Ambiguity in Output

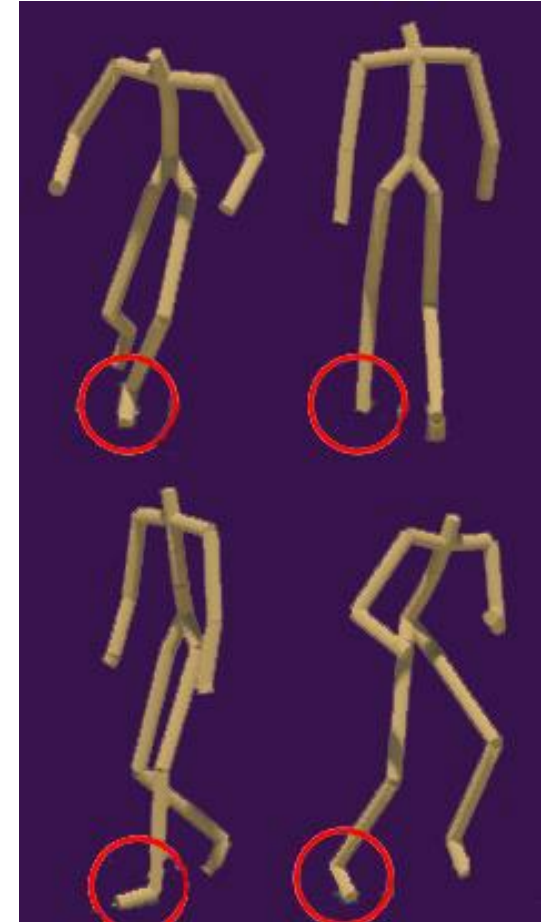
What Happened?

- **Ambiguity:** the same control signal maps to multiple motions
- These motions are averaged in the output



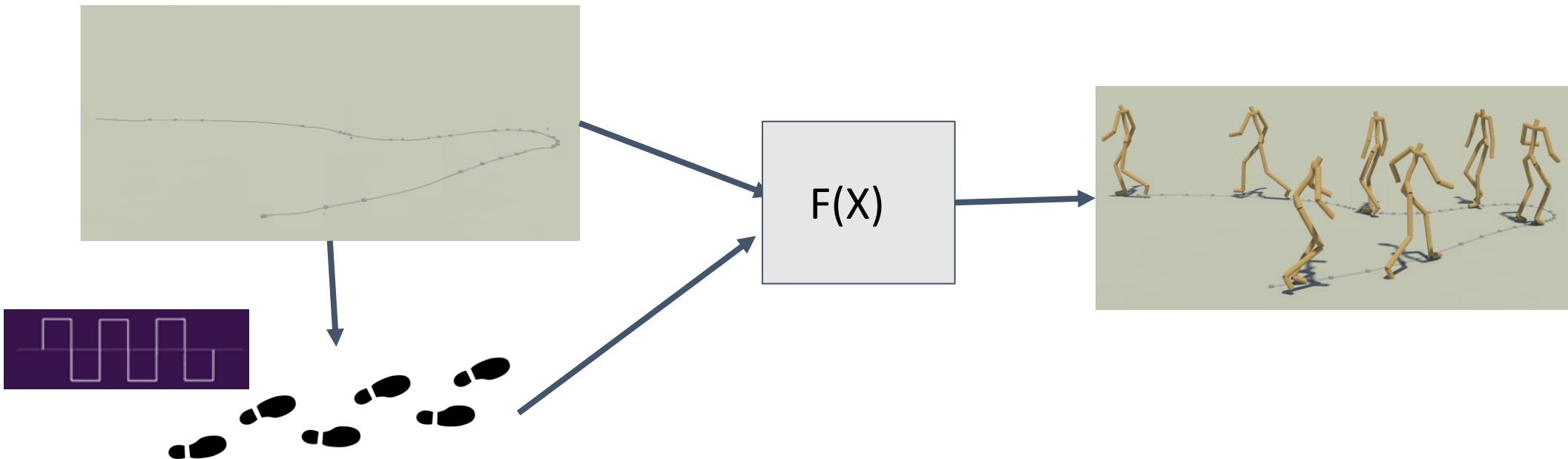
Foot Contact

- Contact times resolve ambiguity
- Automatically label using foot speed and height
- Learn model that generates contact times from trajectory



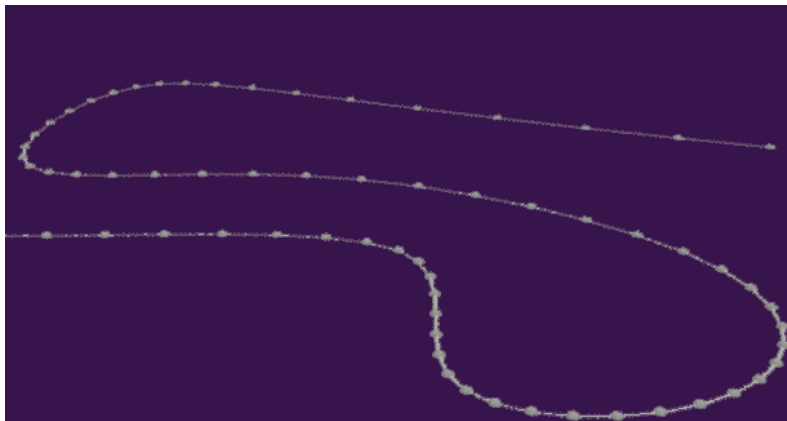
Foot Contact

- Use small neural network to map trajectories to contact durations and frequencies
- Produce timings from durations and frequencies

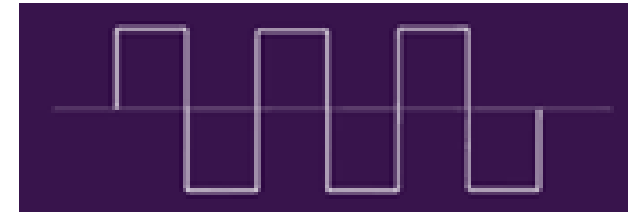


Foot Contact

- Use small neural network to map trajectories to contact durations and frequencies
- Produce timings from durations and frequencies



Trajectory

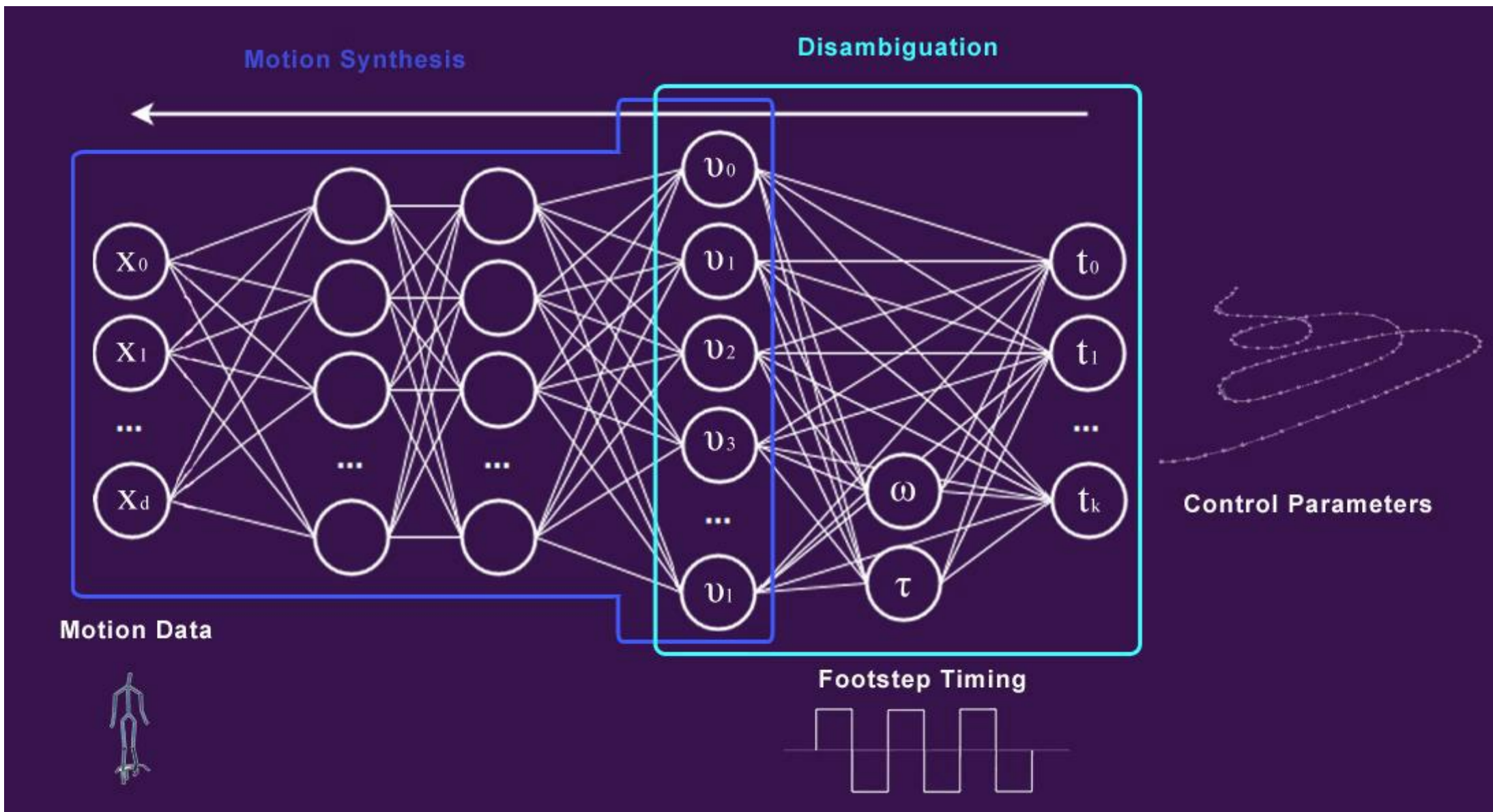


Contact Timings

Contact Durations

Contact Frequencies

Overview





Feedforward Network Output

Motion Editing

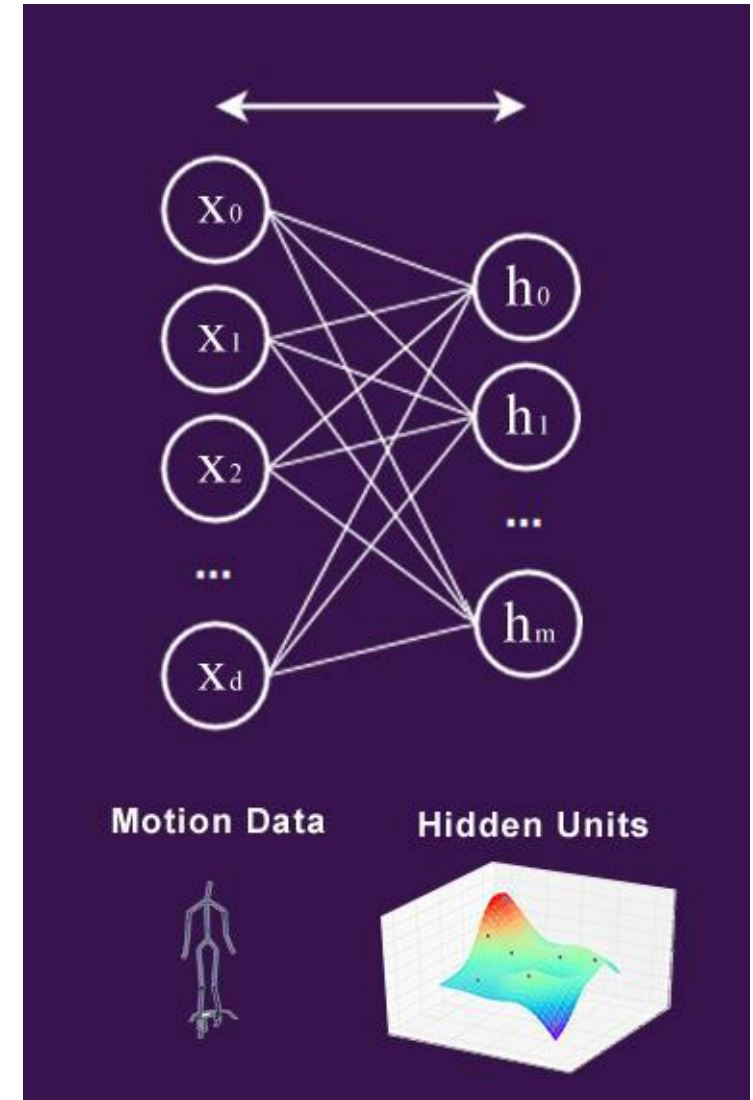
- Once motion is generated it must be edited
- Post processing may not ensure naturalness



Post Processed by Inverse Kinematics

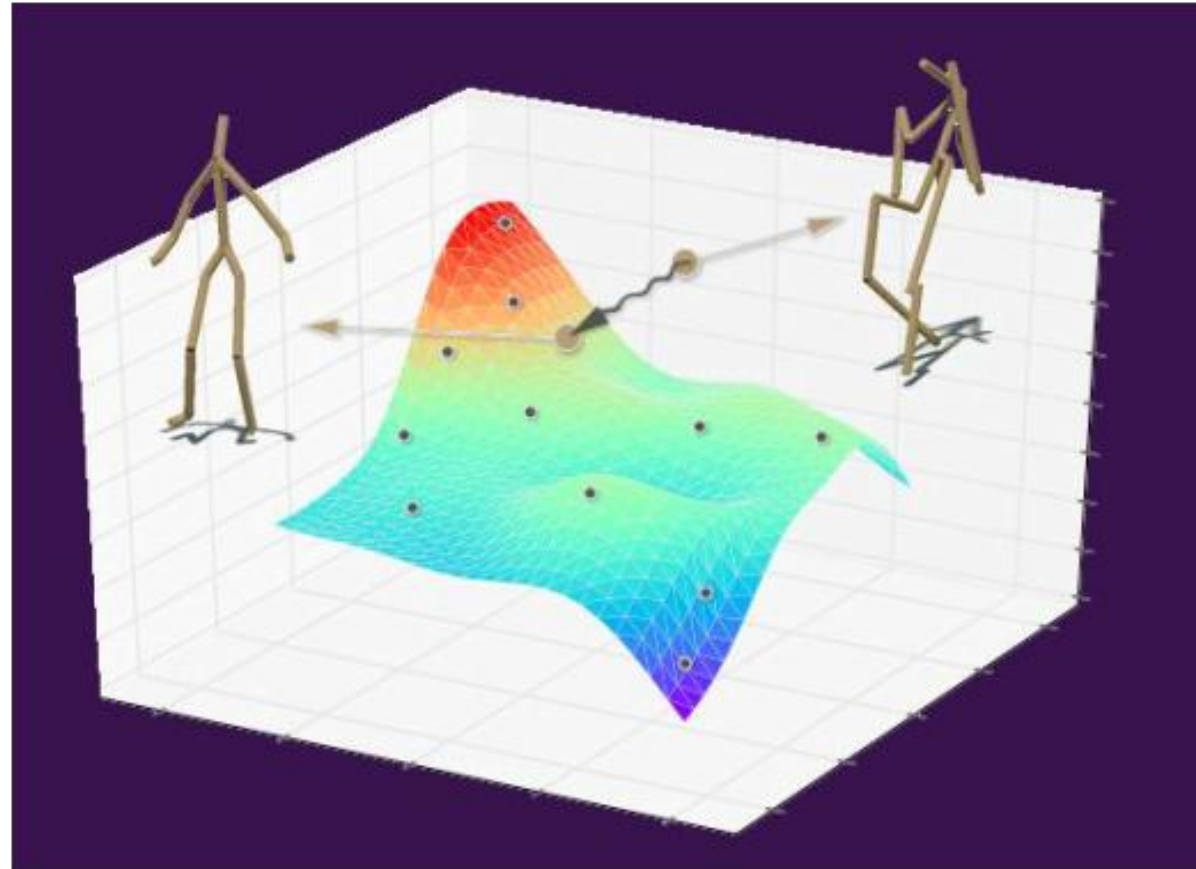
Motion Editing

We edit using the motion manifold learned by a Convolutional Autoencoding Network described earlier



Autoencoder

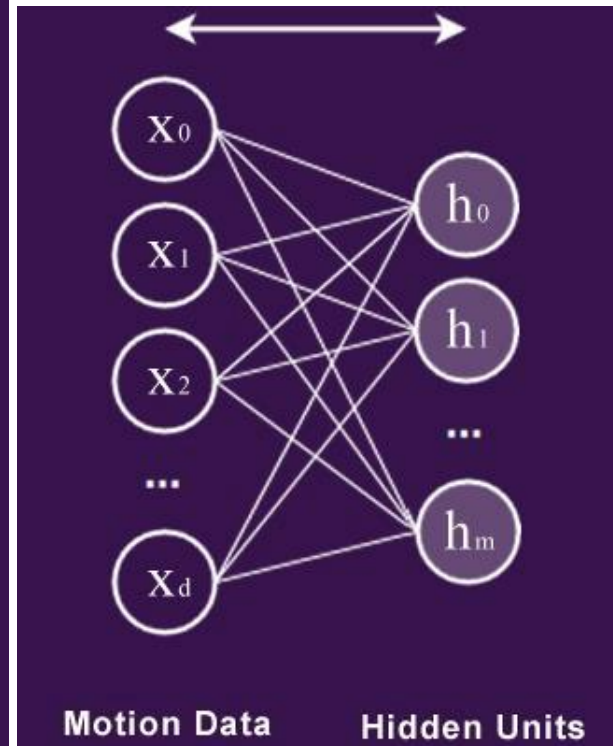
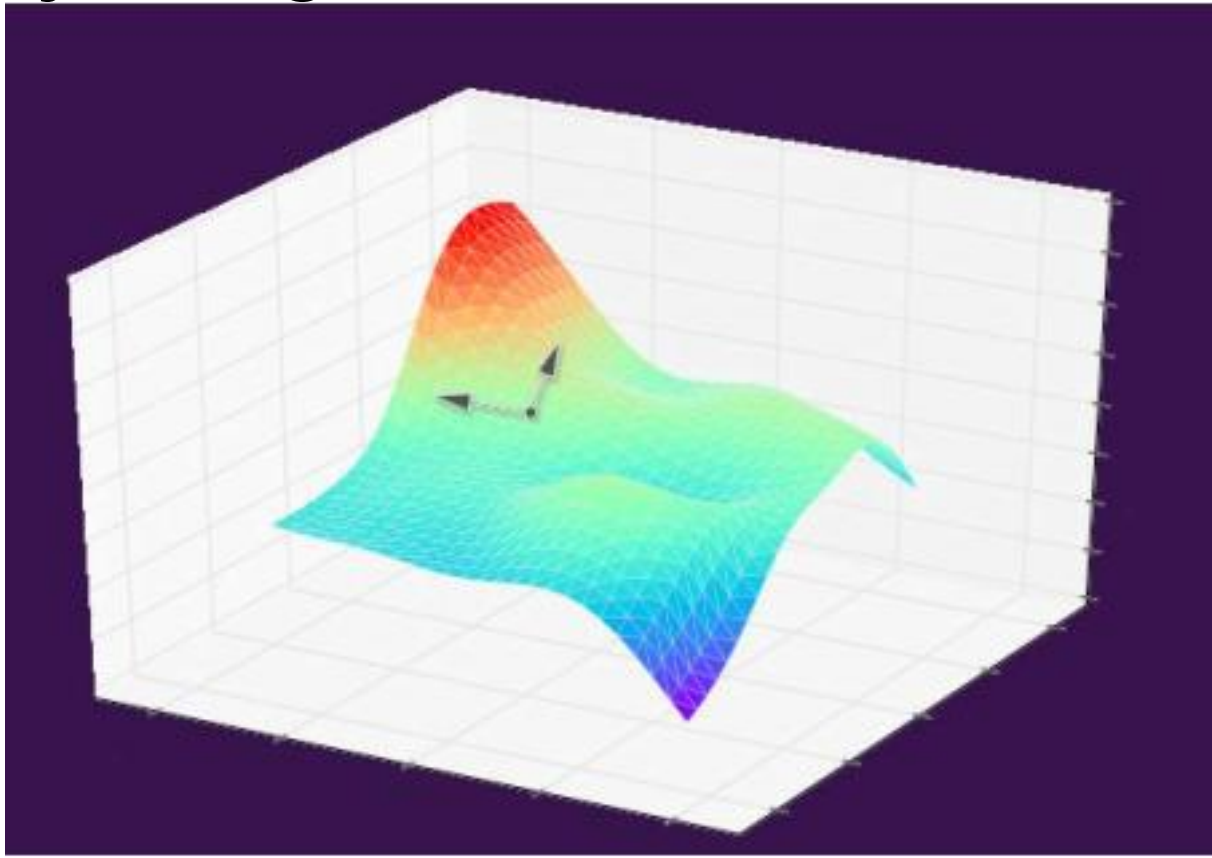
- Learns *projection operator* of motion manifold



Manifold Surface

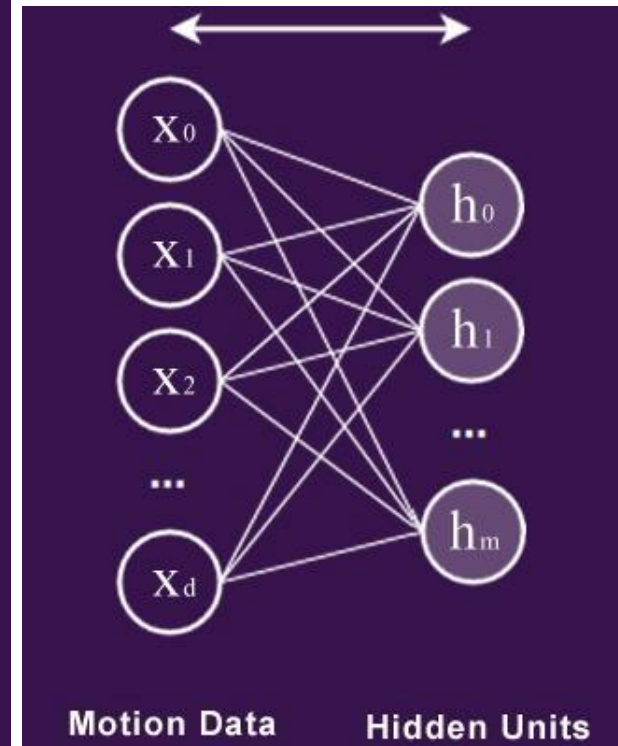
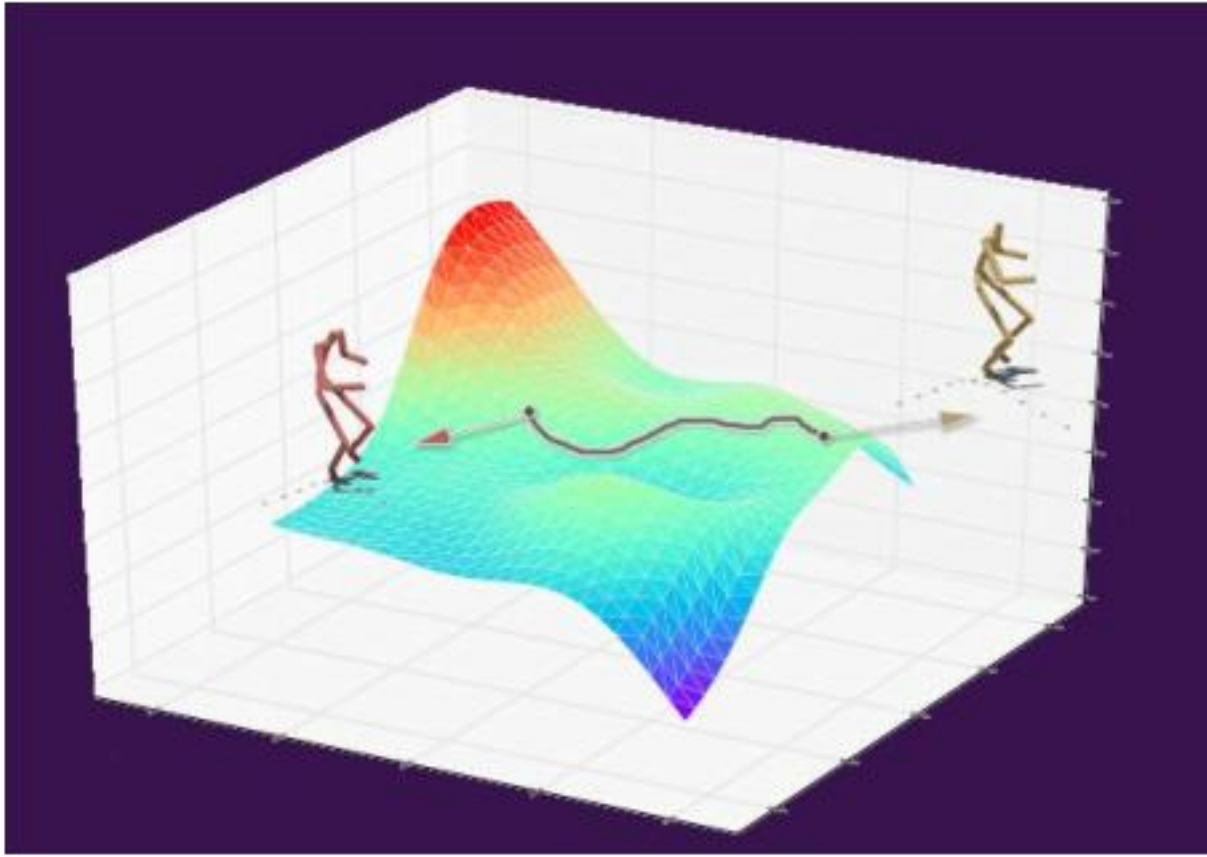
Hidden Unit values parametrise manifold surface

- Adjusting them ensures motion remains natural



Constraint Satisfaction

Motion editing is a *constraint satisfaction problem over Hidden Units*



Constraint Satisfaction

Local foot velocity must equal global velocity
(static with respect to the world)

$$Pos(H) = \sum_j v_r^H + \omega^H \times p_j^H + v_j^H - v'_j$$

v'_j : target velocity of joint j

v_r^H : root velocity

p_j^H, v_j^H joint j 's local position and velocity

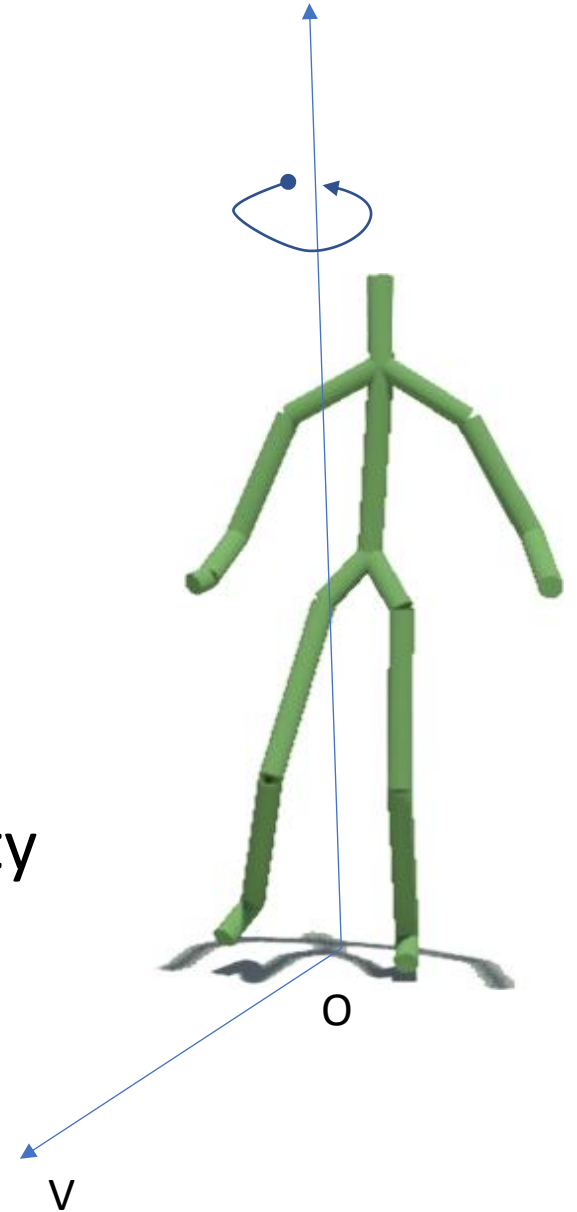
ω^H : body's angular velocity around the Y axis

Output trajectory must equal input trajectory

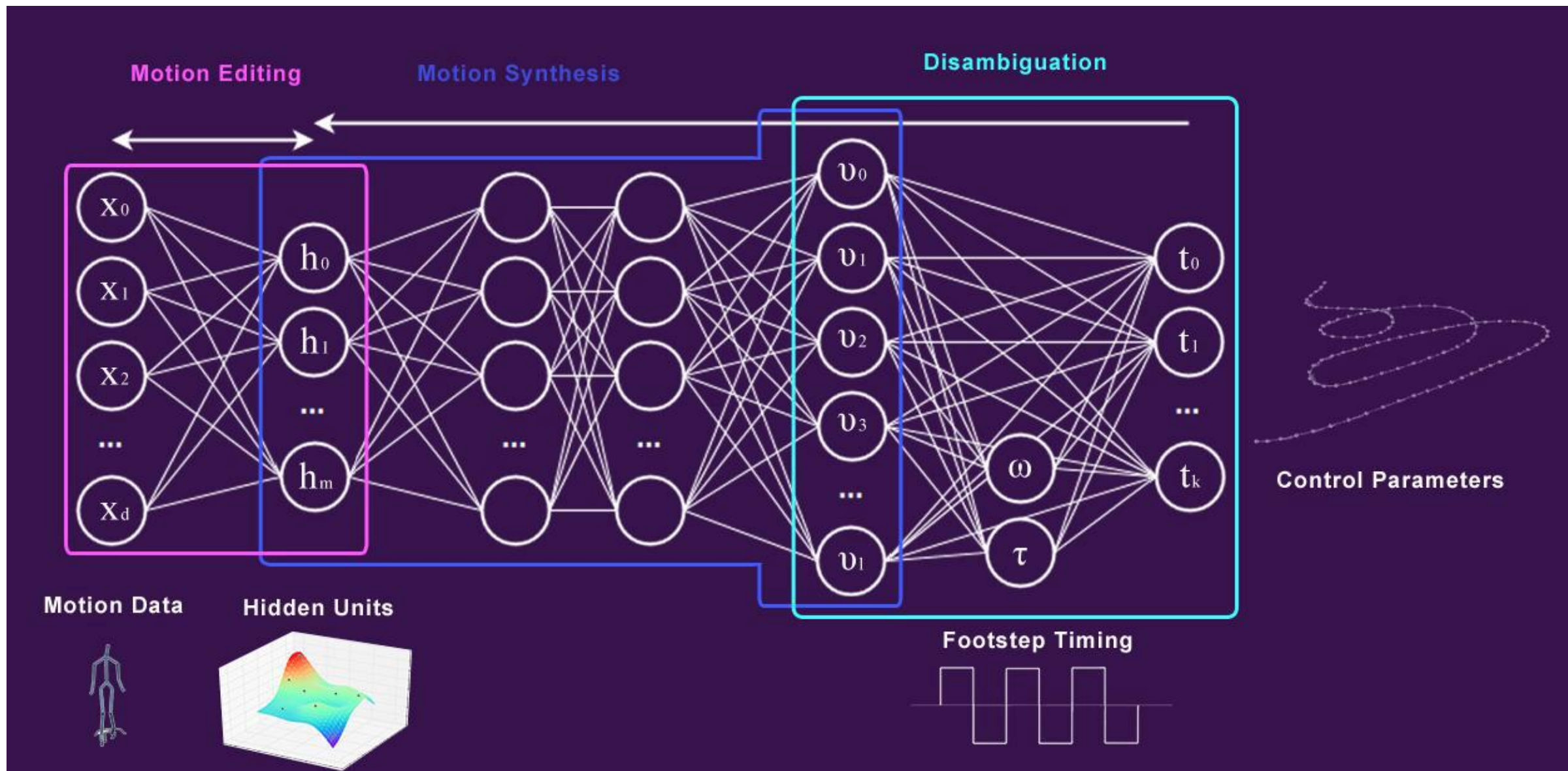
$$Traj(H) = \|\omega^H - \omega'\|_2^2 + \|v_r^H - v'_r\|_2^2$$

Training

- Full CMU database is used (~10 hours of data).
- Separated into overlapping windows of 160 frames and sub-sampled to 30fps.
- Using Joint Positions relative to the root joint.
- Global translation on XZ plane and global rotation on Y axis are removed.
- Rotational velocity around Y axis and translational velocity on XZ plane (relative to forward direction) are added.



Overview

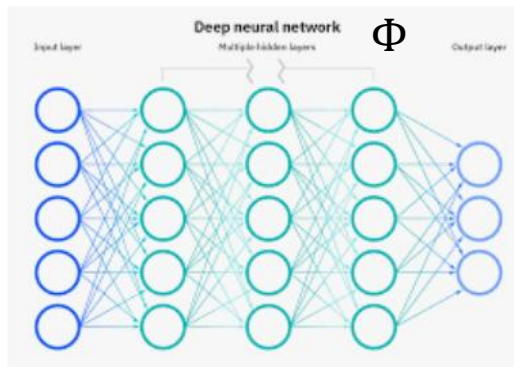




Our Method

A Neural Algorithm of Artistic Style

- Combine style of one image with content of another [Gatys et al. 2015]

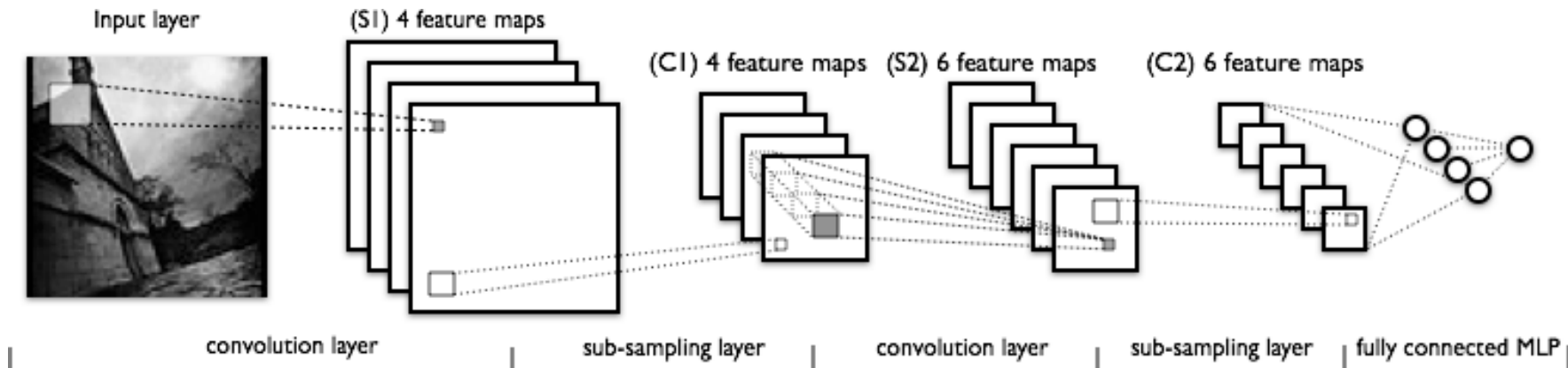


H: hidden units



Convolutional Neural Networks

- *Convolutional Neural Networks* (CNNs) convolve over dimensions with local coherence.
- Great successes in Image Classification, Video Classification, Speech Recognition.
- Convolutional neural networks are those where the network weights are learned in the form of convolutional filters that slide over the image.



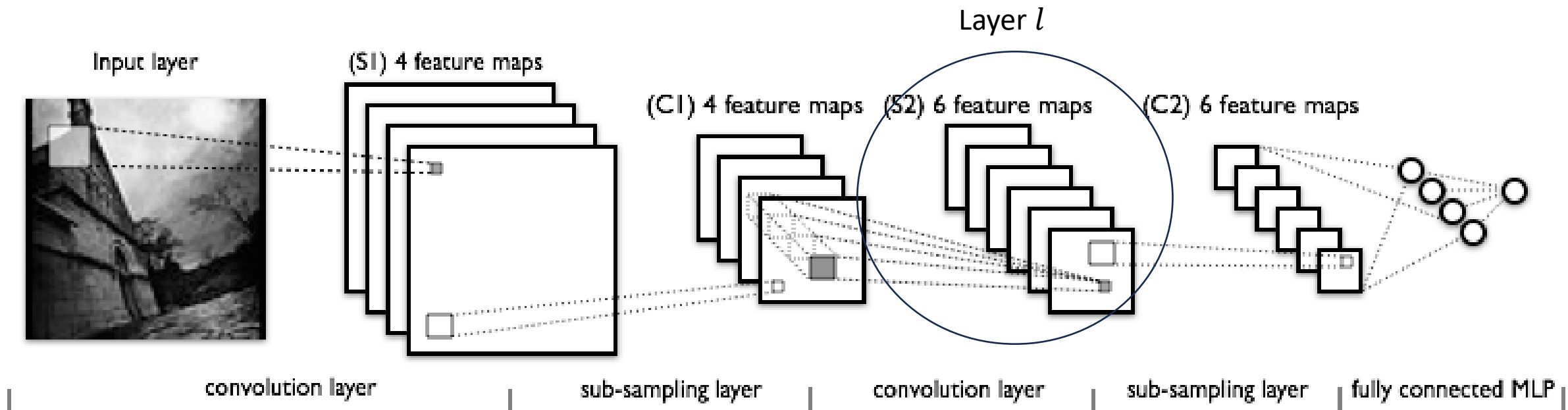
Gram Matrix

The CNN at layer l has N_l feature maps.

Each feature map has M_l pixels, which is its height times its width.

The feature map can be vectorized as: $F^l \in R^{N_l \times M_l}$

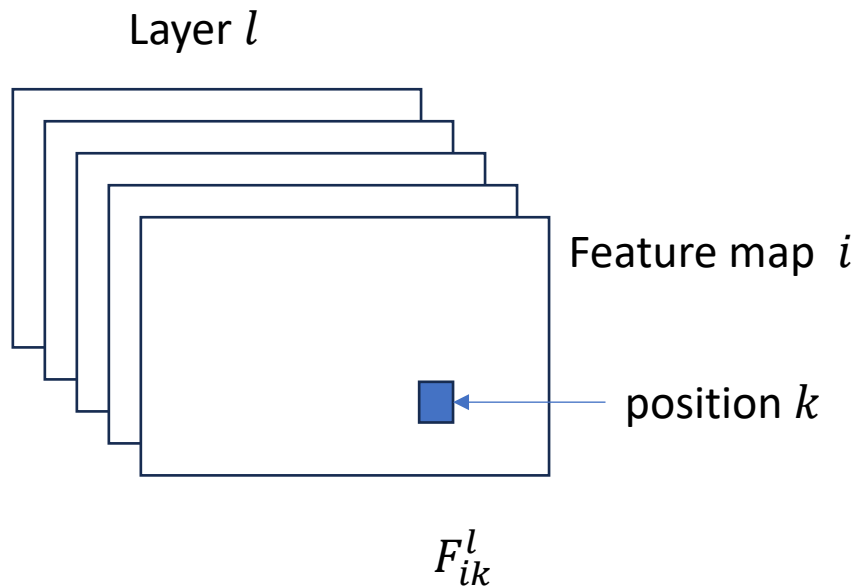
Let's define F_{ik}^l as the activation of the i -th filter at position k



Gram Matrix

The feature map can be vectorized as: $F^l \in R^{N_l \times M_l}$

Let's define F_{ik}^l as the activation of the i -th feature map at position k

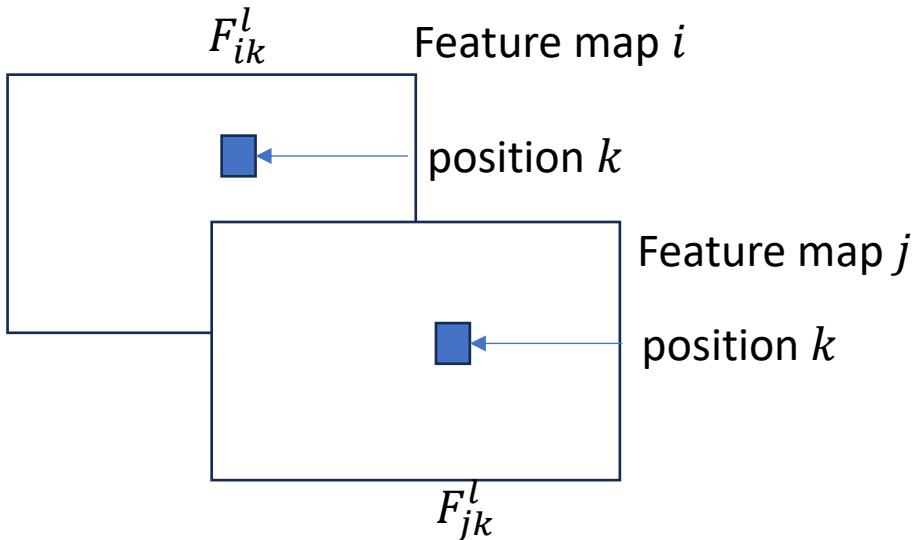


Gram Matrix

- For each feature map pair in layer l , we can compute the Gram matrix $G^l \in R^{N_l \times N_l}$, that represents the co-activation of feature map i and j .

- $G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l$

- This represents the coactivation of the feature map



$$G^l = \left[\begin{array}{ccc} & \overbrace{\dots}^{N_l} & \\ \vdots & \ddots & \vdots \\ & \dots & \end{array} \right]_{N_l}$$

Content Constraint

- Actual Values of *Feature map* encode content
- Optimize an image to make the content same as content image C

$$L_{content} = \sum_{i,k} (F_{ik}^l - C_{ik}^l)^2$$



C

Style Constraint

Optimize the style of the image to be similar to the style image S by minimizing the style loss based on Gram Matrix

$$L_{Style} = \sum_{i,k} (G_{ik}^l - S_{ik}^l)^2$$



S

Producing a stylized image



C



S

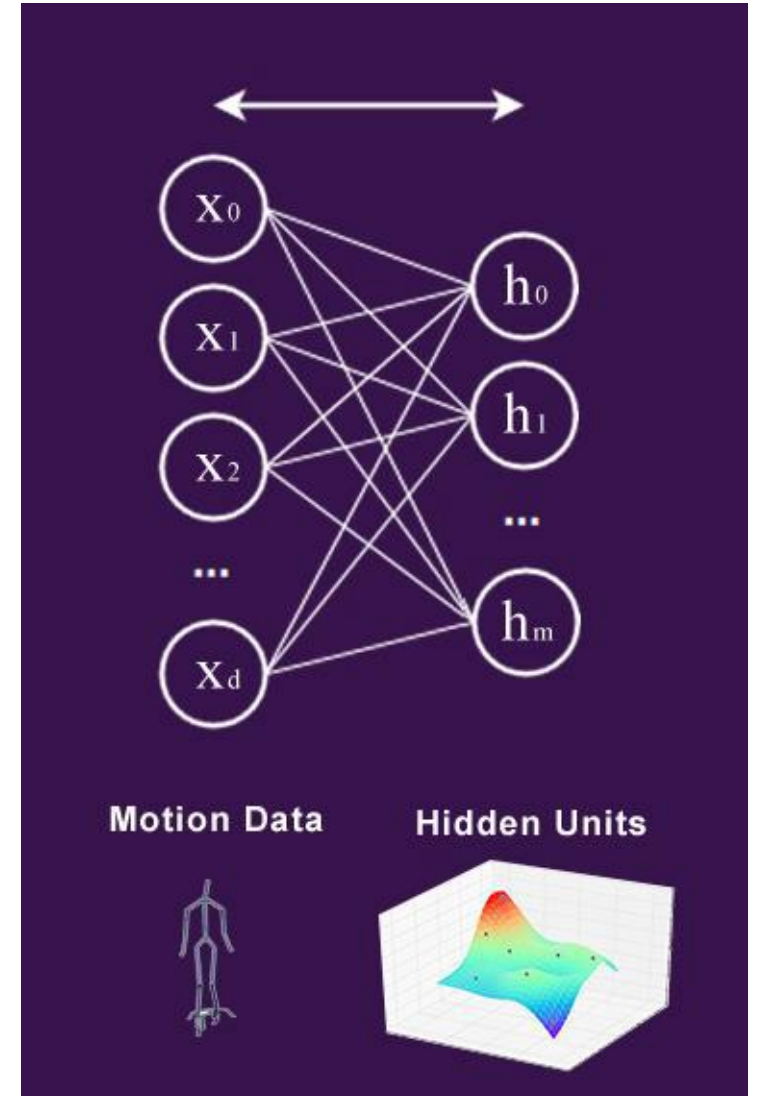
$L_{content} + L_{style}$



A Neural Algorithm of Artistic Style



H: hidden units





Style - Zombie

Discussions

- Motion Matching is an algorithm useful for keeping the motion quality high – as simply replaying the motion will surely appear natural
 - But the search can be costly – also the memory required is intensive (need to keep all the motion in the database)
- Deep learning methods can smooth-out the motion – especially when there is an ambiguity in the output
 - Adding more control signals in the input to reduce the ambiguity is a good idea
 - Could also do adversarial training

Discussions 2

- Using temporal convolution to map the trajectories to the full body motion
- Human locomotion requires resolving the ambiguity for inference
- Combining with the autoencoder allows post-processing the motion in the motion manifold
- This results in realistic motion editing!
- We can also apply motion stylization techniques

Readings

- Daniel Holden, Jun Saito, Taku Komura ["A deep learning framework for character motion synthesis and editing"](#), ACM Transactions on Graphics 35(4), (Proceedings of SIGGRAPH 2016)
- Daniel Holden, Jun Saito, Taku Komura, Tom Joyce ["Learning Motion Manifolds with Convolutional Autoencoders"](#), ACM SIGGRAPH Asia 2015, Technical Briefs
- Mukai, Tomohiko, and Shigeru Kuriyama. "Geostatistical motion interpolation." *ACM SIGGRAPH 2005 Papers*. 2005. 1062-1070.
- [Leon A. Gatys](#), [Alexander S. Ecker](#), [Matthias Bethge](#), **A Neural Algorithm of Artistic Style**
- Clavet 2016, "Motion Matching and The Road to Next-Gen Animation"
- Holden et al. 2020, "Learned Motion Matching"